

Arquitectura Horizontal

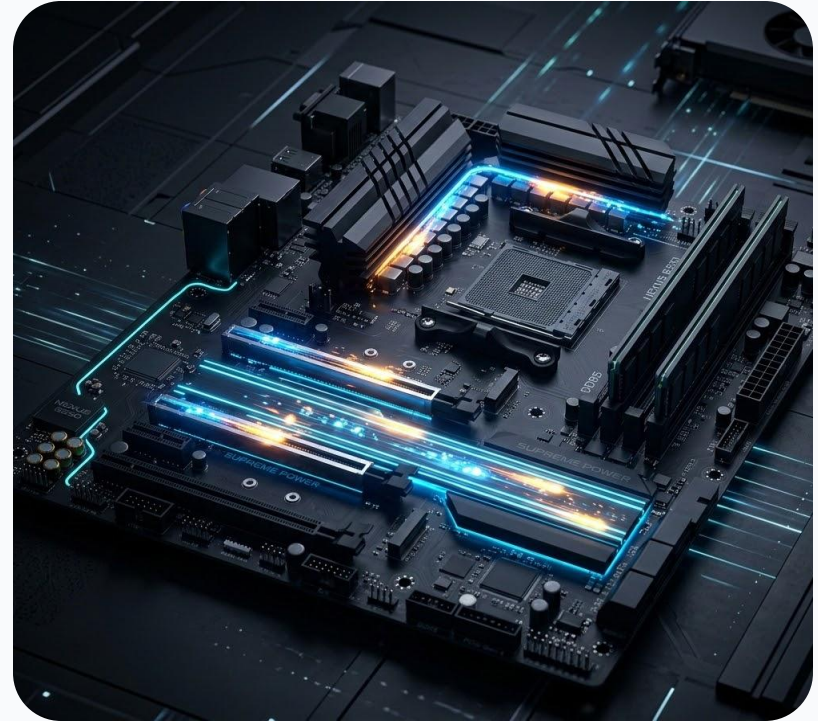
Trayectoria de ejecución



Fundamentos de Arquitectura de computadoras

Análisis de la trayectoria de ejecución horizontal y el flujo de datos dentro del procesador.

- Set de instrucciones
- Ejemplos
- Flujo de datos



Máquina multinivel

NIVEL 5

Nivel de lenguaje orientado al problema

Traducción (Compilador)

NIVEL 4

Nivel de lenguaje ensamblador

Traducción (Ensamblador)

NIVEL 3

Nivel de máquina del sistema operativo

Interpretación parcial (S.O.)

NIVEL 2

Nivel de máquina convencional

Interpretación o Ejecución Directa

NIVEL 1

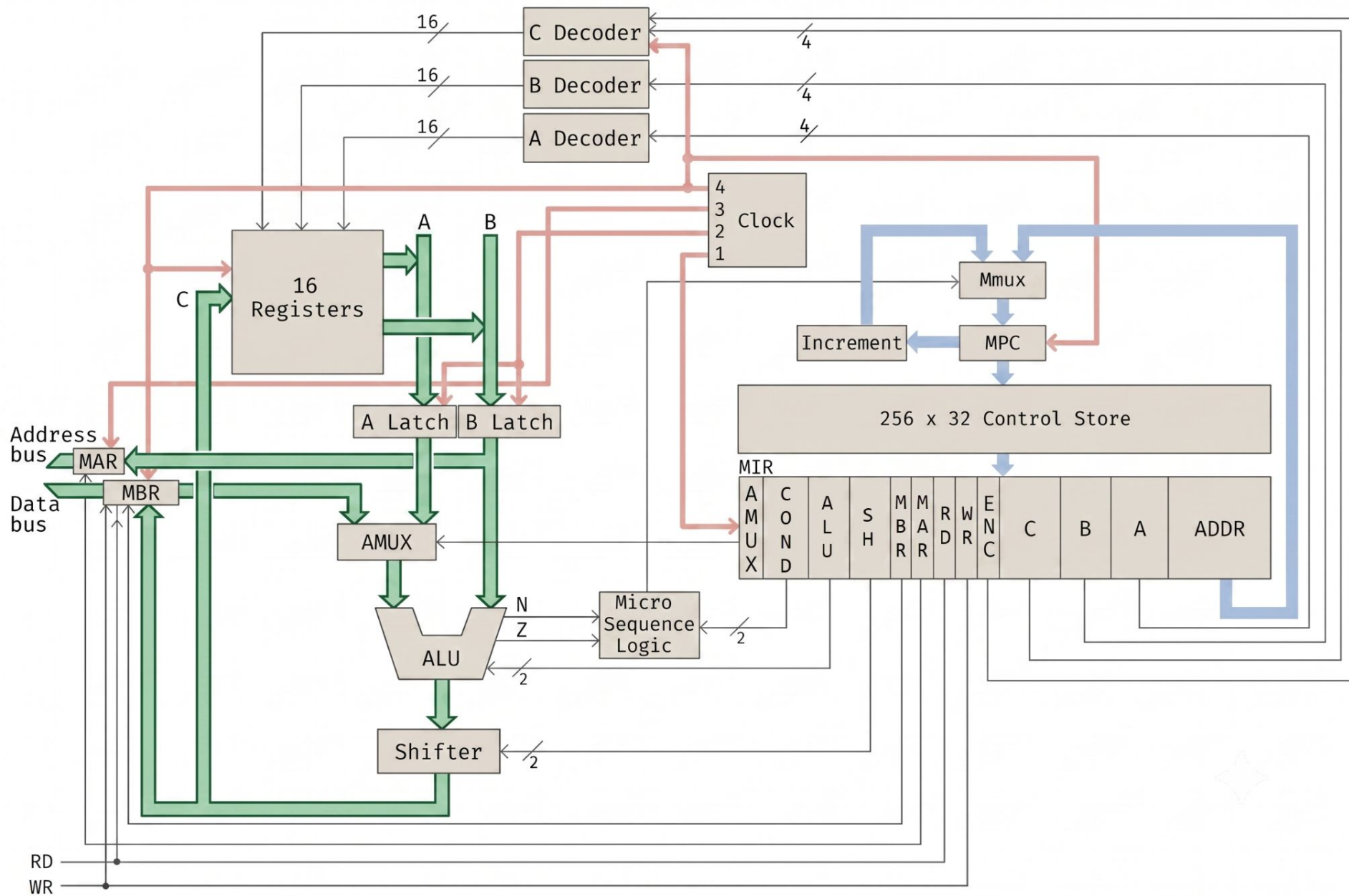
Nivel de microarquitectura

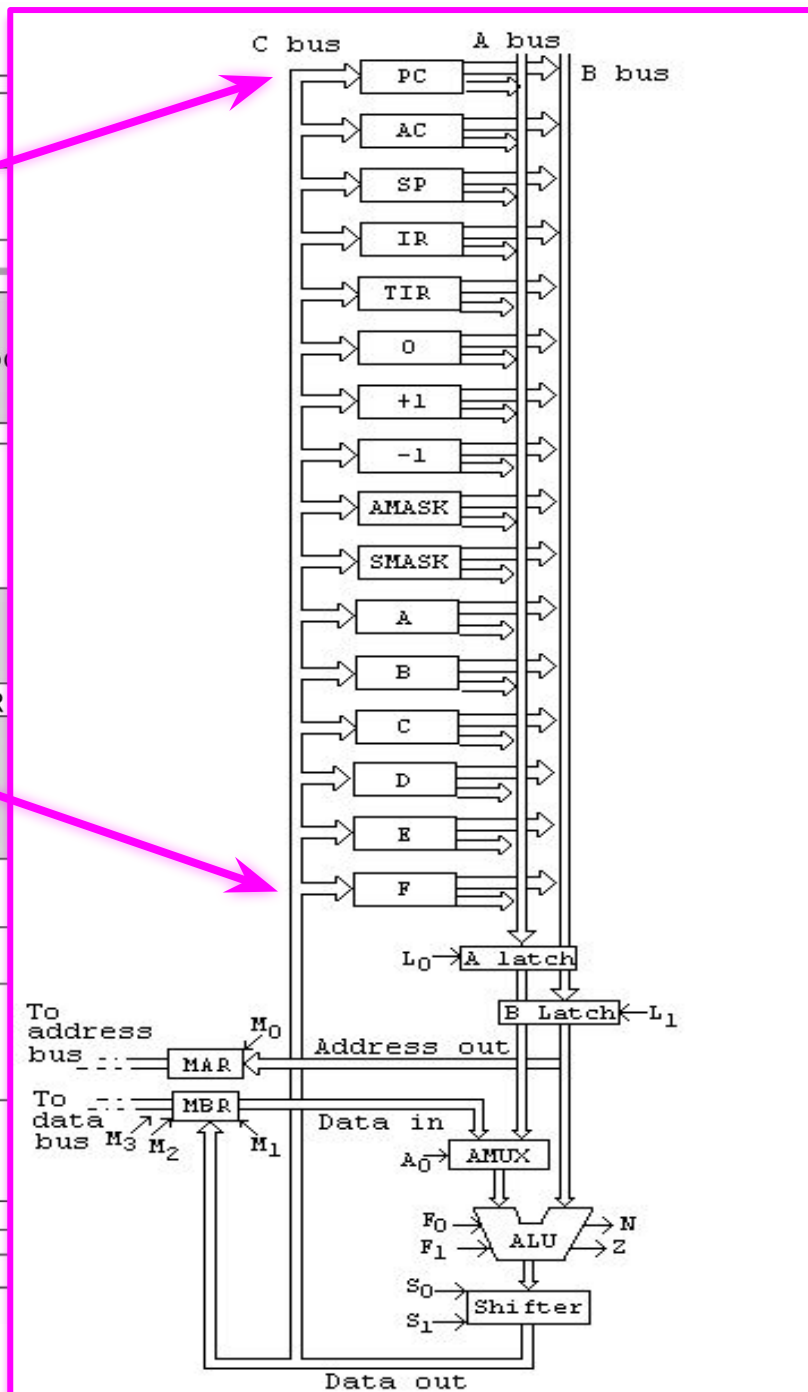
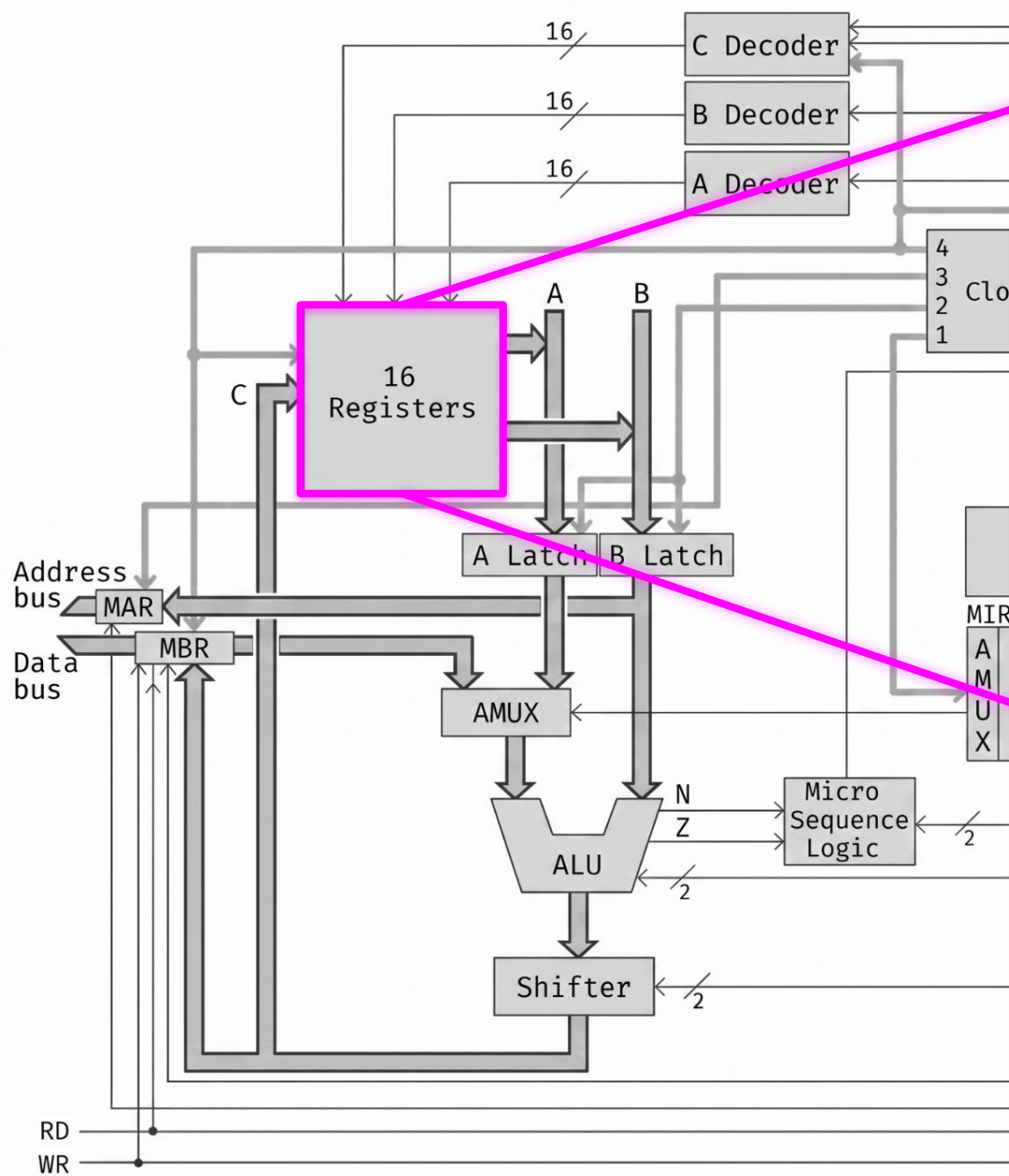
Ejecución directa por el Hardware

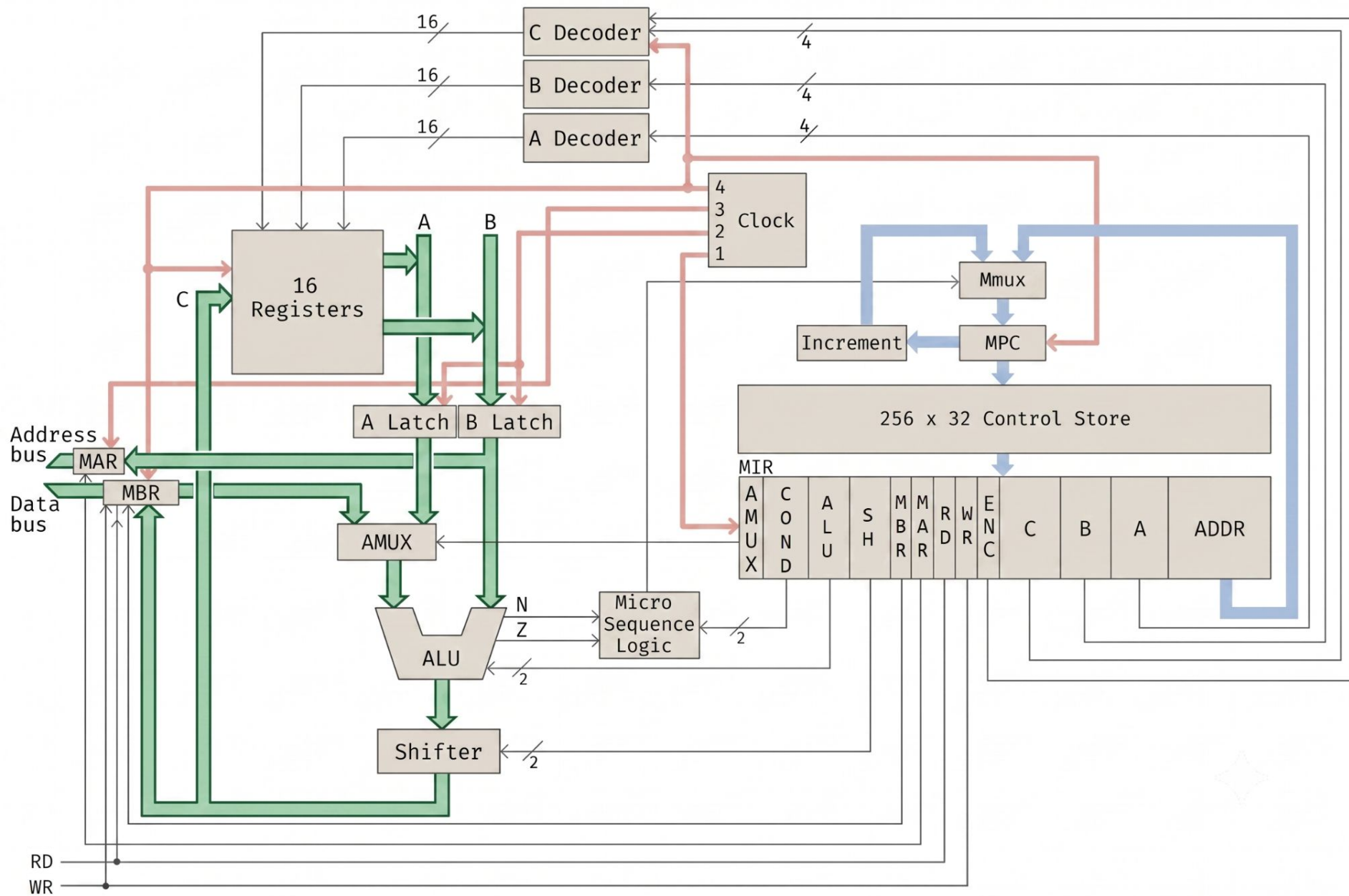
NIVEL 0

Nivel de lógica digital









Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	$ac := m[x]$
0001xxxxxxxxxxxx	STOD	Store direct	$m[x] := ac$
0010xxxxxxxxxxxx	ADDD	Add direct	$ac := ac + m[x]$
0011xxxxxxxxxxxx	SUBD	Subtract direct	$ac := ac - m[x]$
0100xxxxxxxxxxxx	JPOS	Jump positive	if $ac \geq 0$ then $pc := x$
0101xxxxxxxxxxxx	JZER	Jump zero	if $ac = 0$ then $pc := x$
0110xxxxxxxxxxxx	JUMP	Jump	$pc := x$
0111xxxxxxxxxxxx	LOCO	Load constant	$ac := x$ ($0 \leq x \leq 4095$)
1000xxxxxxxxxxxx	LODL	Load local	$ac := m[sp+x]$
1001xxxxxxxxxxxx	STOL	Store local	$m[x+sp] := ac$
1010xxxxxxxxxxxx	ADDL	Add local	$ac := ac + m[sp+x]$
1011xxxxxxxxxxxx	SUBL	Subtract local	$ac := ac - m[sp+x]$
1100xxxxxxxxxxxx	JNEG	Jump negative	if $ac < 0$ then $pc := x$
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if $ac \neq 0$ then $pc := x$
1110xxxxxxxxxxxx	CALL	Call procedure	$sp := sp - 1$; $m[sp] := pc$; $pc := x$
1111000000000000	PSHI	Push indirect	$sp := sp - 1$; $m[sp] := m[ac]$
1111001000000000	POPI	Pop indirect	$m[ac] := m[sp]$; $sp := sp + 1$
1111010000000000	PUSH	Push onto stack	$sp := sp - 1$; $m[sp] := ac$
1111011000000000	POP	Pop from stack	$ac := m[sp]$; $sp := sp + 1$
1111100000000000	RETN	Return	$pc := m[sp]$; $sp := sp + 1$
1111101000000000	SWAP	Swap ac sp	$tmp := ac$; $ac := sp$; $sp := tmp$
11111100yyyyyyyy	INSP	Increment sp	$sp := sp + y$ ($0 \leq y \leq 255$)
11111110yyyyyyyy	DESP	Decrement sp	$sp := sp - y$ ($0 \leq y \leq 255$)

Conjunto de instrucciones
para programar
la aplicación

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxx	LODD	Load direct	$ac := m[x]$
0001xxxxxxxxxx	STOD	Store direct	$m[x] := ac$
0010xxxxxxxxxx	ADDD	Add direct	$ac := ac + m[x]$
0011xxxxxxxxxx	SUBD	Subtract direct	$ac := ac - m[x]$
0100xxxxxxxxxx	JPOS	Jump positive	if $ac \geq 0$ then $pc := x$
0101xxxxxxxxxx	JZER	Jump zero	if $ac = 0$ then $pc := x$
0110xxxxxxxxxx	JUMP	Jump	$pc := x$
0111xxxxxxxxxx	LOCO	Load constant	$ac := x$ ($0 \leq x \leq 4095$)
1000xxxxxxxxxx	LODL	Load local	$ac := m[sp+x]$
1001xxxxxxxxxx	STOL	Store local	$m[x+sp] := ac$
1010xxxxxxxxxx	ADDL	Add local	$ac := ac + m[sp+x]$
1011xxxxxxxxxx	SUBL	Subtract local	$ac := ac - m[sp+x]$
1100xxxxxxxxxx	JNEG	Jump negative	if $ac < 0$ then $pc := x$
1101xxxxxxxxxx	JNZE	Jump nonzero	if $ac \neq 0$ then $pc := x$
1110xxxxxxxxxx	CALL	Call procedure	$sp := sp - 1$; $m[sp] := pc$; $pc := x$
1111000000000000	PSHI	Push indirect	$sp := sp - 1$; $m[sp] := m[ac]$
1111001000000000	POPI	Pop indirect	$m[ac] := m[sp]$; $sp := sp + 1$
1111010000000000	PUSH	Push onto stack	$sp := sp - 1$; $m[sp] := ac$
1111011000000000	POP	Pop from stack	$ac := m[sp]$; $sp := sp + 1$
1111100000000000	RETN	Return	$pc := m[sp]$; $sp := sp + 1$
1111101000000000	SWAP	Swap ac sp	$tmp := ac$; $ac := sp$; $sp := tmp$
11111100yyyyyyyy	INSP	Increment sp	$sp := sp + y$ ($0 \leq y \leq 255$)
11111110yyyyyyyy	DESP	Decrement sp	$sp := sp - y$ ($0 \leq y \leq 255$)

Conjunto de instrucciones
para programar
la aplicación

15 instr. con operando de 12 bits

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxx	LODD	Load direct	$ac := m[x]$
0001xxxxxxxxxx	STOD	Store direct	$m[x] := ac$
0010xxxxxxxxxx	ADDD	Add direct	$ac := ac + m[x]$
0011xxxxxxxxxx	SUBD	Subtract direct	$ac := ac - m[x]$
0100xxxxxxxxxx	JPOS	Jump positive	if $ac \geq 0$ then $pc := x$
0101xxxxxxxxxx	JZER	Jump zero	if $ac = 0$ then $pc := x$
0110xxxxxxxxxx	JUMP	Jump	$pc := x$
0111xxxxxxxxxx	LOCO	Load constant	$ac := x$ ($0 \leq x \leq 4095$)
1000xxxxxxxxxx	LODL	Load local	$ac := m[sp+x]$
1001xxxxxxxxxx	STOL	Store local	$m[x+sp] := ac$
1010xxxxxxxxxx	ADDL	Add local	$ac := ac + m[sp+x]$
1011xxxxxxxxxx	SUBL	Subtract local	$ac := ac - m[sp+x]$
1100xxxxxxxxxx	JNEG	Jump negative	if $ac < 0$ then $pc := x$
1101xxxxxxxxxx	JNZE	Jump nonzero	if $ac \neq 0$ then $pc := x$
1110xxxxxxxxxx	CALL	Call procedure	$sp := sp - 1$; $m[sp] := pc$; $pc := x$
1111000000000000	PSHI	Push indirect	$sp := sp - 1$; $m[sp] := m[ac]$
1111001000000000	POPI	Pop indirect	$m[ac] := m[sp]$; $sp := sp + 1$
1111010000000000	PUSH	Push onto stack	$sp := sp - 1$; $m[sp] := ac$
1111011000000000	POP	Pop from stack	$ac := m[sp]$; $sp := sp + 1$
1111100000000000	RETN	Return	$pc := m[sp]$; $sp := sp + 1$
1111101000000000	SWAP	Swap ac sp	$tmp := ac$; $ac := sp$; $sp := tmp$
11111100yyyyyyy	INSP	Increment sp	$sp := sp + y$ ($0 \leq y \leq 255$)
11111110yyyyyyy	DESP	Decrement sp	$sp := sp - y$ ($0 \leq y \leq 255$)

Conjunto de instrucciones
para programar
la aplicación

15 instr. con operando de 12 bits

6 instrucciones sin operando

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxx	LODD	Load direct	$ac := m[x]$
0001xxxxxxxxxx	STOD	Store direct	$m[x] := ac$
0010xxxxxxxxxx	ADDD	Add direct	$ac := ac + m[x]$
0011xxxxxxxxxx	SUBD	Subtract direct	$ac := ac - m[x]$
0100xxxxxxxxxx	JPOS	Jump positive	if $ac \geq 0$ then $pc := x$
0101xxxxxxxxxx	JZER	Jump zero	if $ac = 0$ then $pc := x$
0110xxxxxxxxxx	JUMP	Jump	$pc := x$
0111xxxxxxxxxx	LOCO	Load constant	$ac := x$ ($0 \leq x \leq 4095$)
1000xxxxxxxxxx	LODL	Load local	$ac := m[sp+x]$
1001xxxxxxxxxx	STOL	Store local	$m[x+sp] := ac$
1010xxxxxxxxxx	ADDL	Add local	$ac := ac + m[sp+x]$
1011xxxxxxxxxx	SUBL	Subtract local	$ac := ac - m[sp+x]$
1100xxxxxxxxxx	JNEG	Jump negative	if $ac < 0$ then $pc := x$
1101xxxxxxxxxx	JNZE	Jump nonzero	if $ac \neq 0$ then $pc := x$
1110xxxxxxxxxx	CALL	Call procedure	$sp := sp - 1$; $m[sp] := pc$; $pc := x$
1111000000000000	PSHI	Push indirect	$sp := sp - 1$; $m[sp] := m[ac]$
1111001000000000	POPI	Pop indirect	$m[ac] := m[sp]$; $sp := sp + 1$
1111010000000000	PUSH	Push onto stack	$sp := sp - 1$; $m[sp] := ac$
1111011000000000	POP	Pop from stack	$ac := m[sp]$; $sp := sp + 1$
1111100000000000	RETN	Return	$pc := m[sp]$; $sp := sp + 1$
1111101000000000	SWAP	Swap ac sp	$tmp := ac$; $ac := sp$; $sp := tmp$
11111100yyyyyyyy	INSP	Increment sp	$sp := sp + y$ ($0 \leq y \leq 255$)
11111110yyyyyyyy	DESP	Decrement sp	$sp := sp - y$ ($0 \leq y \leq 255$)

Conjunto de instrucciones
para programar
la aplicación

15 instr. con operando de 12 bits

6 instrucciones sin operando

2 instrucciones con operando de 8 bits

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxxx	JPOS	Jump positive	if ac>=0 then pc:=x
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxxxxxxxxxx	LOCO	Load constant	ac:=x (0<=x<=4095)
1000xxxxxxxxxxxx	LODL	Load local	ac:=m[sp+x]
1001xxxxxxxxxxxx	STOL	Store local	m[x+sp]:=ac
1010xxxxxxxxxxxx	ADDL	Add local	ac:=ac+m[sp+x]
1011xxxxxxxxxxxx	SUBL	Subtract local	ac:=ac-m[sp+x]
1100xxxxxxxxxxxx	JNEG	Jump negative	if ac<0 then pc:=x
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if ac<>0 then pc:=x
1110xxxxxxxxxxxx	CALL	Call procedure	sp:=sp-1; m[sp]:=pc; pc:=x
1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac
1111011000000000	POP	Pop from stack	ac:=m[sp]; sp:=sp+ 1
1111100000000000	RETN	Return	pc:=m[sp]; sp:=sp+ 1
1111101000000000	SWAP	Swap ac sp	tmp:=ac; ac:=sp; sp:=tmp
11111100yyyyyyyy	INSP	Increment sp	sp:=sp+y (0<=y<=255)
11111110yyyyyyyy	DESP	Decrement sp	sp:=sp-y (0<=y<=255)

STORE DIRECT

...
STOD 9
...

Guardar el valor del AC en la memoria(9)

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxxx	JPOS	Jump positive	if ac>=0 then pc:=x
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxxxxxxxxxx	LOCO	Load constant	ac:=x (0<=x<=4095)
1000xxxxxxxxxxxx	LODL	Load local	ac:=m[sp+x]
1001xxxxxxxxxxxx	STOL	Store local	m[x+sp]:=ac
1010xxxxxxxxxxxx	ADDL	Add local	ac:=ac+m[sp+x]
1011xxxxxxxxxxxx	SUBL	Subtract local	ac:=ac-m[sp+x]
1100xxxxxxxxxxxx	JNEG	Jump negative	if ac<0 then pc:=x
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if ac<>0 then pc:=x
1110xxxxxxxxxxxx	CALL	Call procedure	sp:=sp-1; m[sp]:=pc; pc:=x
1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac
1111011000000000	POP	Pop from stack	ac:=m[sp]; sp:=sp+ 1
1111100000000000	RETN	Return	pc:=m[sp]; sp:=sp+ 1
1111101000000000	SWAP	Swap ac sp	tmp:=ac; ac:=sp; sp:=tmp
11111100yyyyyyyy	INSP	Increment sp	sp:=sp+y (0<=y<=255)
11111110yyyyyyyy	DESP	Decrement sp	sp:=sp-y (0<=y<=255)

STORE DIRECT

...
STOD 9
...

Guardar el valor del AC en la memoria(9)

m[x] := ac

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxxx	JPOS	Jump positive	if ac>=0 then pc:=x
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxxxxxxxxxx	LOCO	Load constant	ac:=x (0<=x<=4095)
1000xxxxxxxxxxxx	LODL	Load local	ac:=m[sp+x]
1001xxxxxxxxxxxx	STOL	Store local	m[x+sp]:=ac
1010xxxxxxxxxxxx	ADDL	Add local	ac:=ac+m[sp+x]
1011xxxxxxxxxxxx	SUBL	Subtract local	ac:=ac-m[sp+x]
1100xxxxxxxxxxxx	JNEG	Jump negative	if ac<0 then pc:=x
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if ac<>0 then pc:=x
1110xxxxxxxxxxxx	CALL	Call procedure	sp:=sp-1; m[sp]:=pc; pc:=x
1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac
1111011000000000	POP	Pop from stack	ac:=m[sp]; sp:=sp+ 1
1111100000000000	RETN	Return	pc:=m[sp]; sp:=sp+ 1
1111101000000000	SWAP	Swap ac sp	tmp:=ac; ac:=sp; sp:=tmp
11111100yyyyyyyy	INSP	Increment sp	sp:=sp+y (0<=y<=255)
11111110yyyyyyyy	DESP	Decrement sp	sp:=sp-y (0<=y<=255)

STORE DIRECT

...
STOD 9
 ...

Guardar el valor del AC en la memoria(9)

m[x] := ac

0001 0000 0000 1001

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	$ac := m[x]$
0001xxxxxxxxxxxx	STOD	Store direct	$m[x] := ac$
0010xxxxxxxxxxxx	ADDD	Add direct	$ac := ac + m[x]$
0011xxxxxxxxxxxx	SUBD	Subtract direct	$ac := ac - m[x]$
0100xxxxxxxxxxxx	JPOS	Jump positive	if $ac \geq 0$ then $pc := x$
0101xxxxxxxxxxxx	JZER	Jump zero	if $ac = 0$ then $pc := x$
0110xxxxxxxxxxxx	JUMP	Jump	$pc := x$
0111xxxxxxxxxxxx	LOCO	Load constant	$ac := x$ ($0 \leq x \leq 4095$)
1000xxxxxxxxxxxx	LODL	Load local	$ac := m[sp+x]$
1001xxxxxxxxxxxx	STOL	Store local	$m[x+sp] := ac$
1010xxxxxxxxxxxx	ADDL	Add local	$ac := ac + m[sp+x]$
1011xxxxxxxxxxxx	SUBL	Subtract local	$ac := ac - m[sp+x]$
1100xxxxxxxxxxxx	JNEG	Jump negative	if $ac < 0$ then $pc := x$
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if $ac \neq 0$ then $pc := x$
1110xxxxxxxxxxxx	CALL	Call procedure	$sp := sp - 1$; $m[sp] := pc$; $pc := x$
1111000000000000	PSHI	Push indirect	$sp := sp - 1$; $m[sp] := m[ac]$
1111001000000000	POPI	Pop indirect	$m[ac] := m[sp]$; $sp := sp + 1$
1111010000000000	PUSH	Push onto stack	$sp := sp - 1$; $m[sp] := ac$
1111011000000000	POP	Pop from stack	$ac := m[sp]$; $sp := sp + 1$
1111100000000000	RETN	Return	$pc := m[sp]$; $sp := sp + 1$
1111101000000000	SWAP	Swap ac sp	$tmp := ac$; $ac := sp$; $sp := tmp$
11111100yyyyyyyy	INSP	Increment sp	$sp := sp + y$ ($0 \leq y \leq 255$)
11111110yyyyyyyy	DESP	Decrement sp	$sp := sp - y$ ($0 \leq y \leq 255$)

STORE DIRECT

...
 STOD 9
 ...

Guardar el valor del AC en la memoria(9)

- $mar := ir$
- $mbr := ac$
- wr

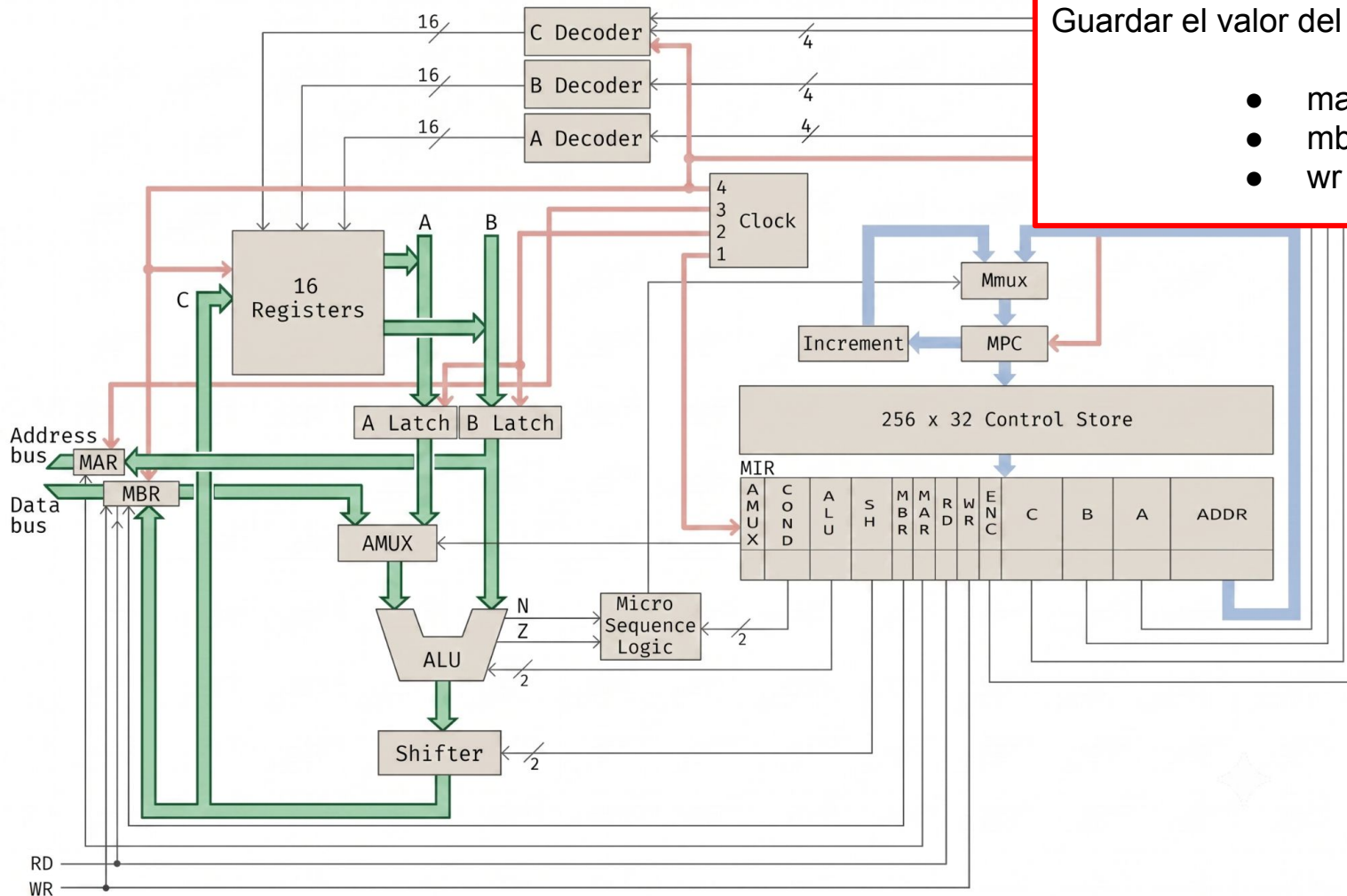
Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxx	LODD	Load direct	$ac := m[x]$
0001xxxxxxxxxx	STOD	Store direct	$m[x] := ac$
0010xxxxxxxxxx	ADDD	Add direct	$ac := ac + m[x]$
0011xxxxxxxxxx	SUBD	Subtract direct	$ac := ac - m[x]$
0100xxxxxxxxxx	JPOS	Jump positive	if $ac \geq 0$ then $pc := x$

STORE DIRECT

...
 STOD 9
 ...

Guardar el valor del AC en la memoria(9)

- $mar := ir$
- $mbr := ac$
- wr



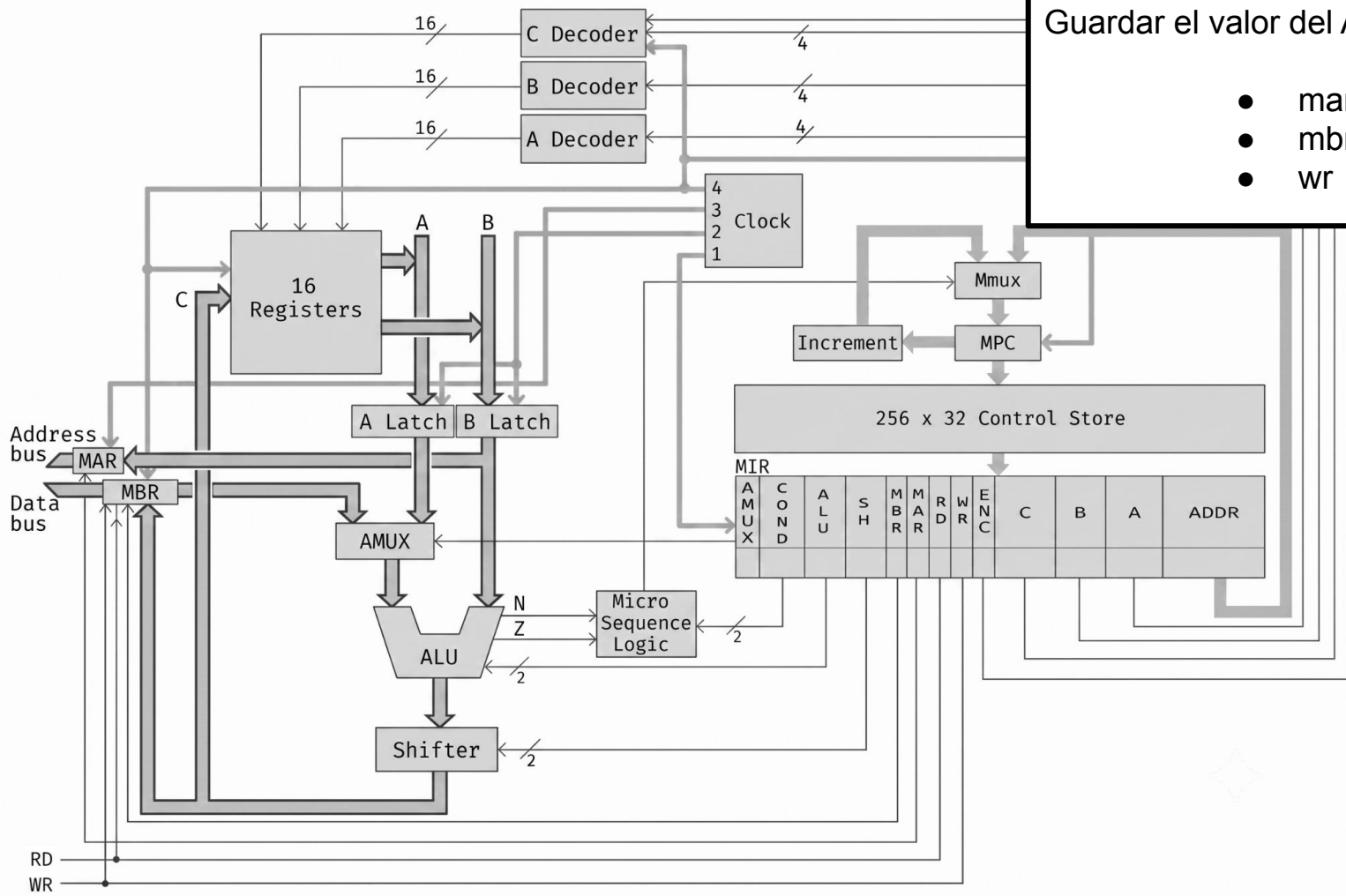
Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxx	LODD	Load direct	$ac := m[x]$
0001xxxxxxxxxx	STOD	Store direct	$m[x] := ac$
0010xxxxxxxxxx	ADDD	Add direct	$ac := ac + m[x]$
0011xxxxxxxxxx	SUBD	Subtract direct	$ac := ac - m[x]$
0100xxxxxxxxxx	JPOS	Jump positive	if $ac \geq 0$ then $pc := x$

STORE DIRECT

...
 STOD 9
 ...

Guardar el valor del AC en la memoria(9)

- $mar := ir$
- $mbr := ac$
- wr



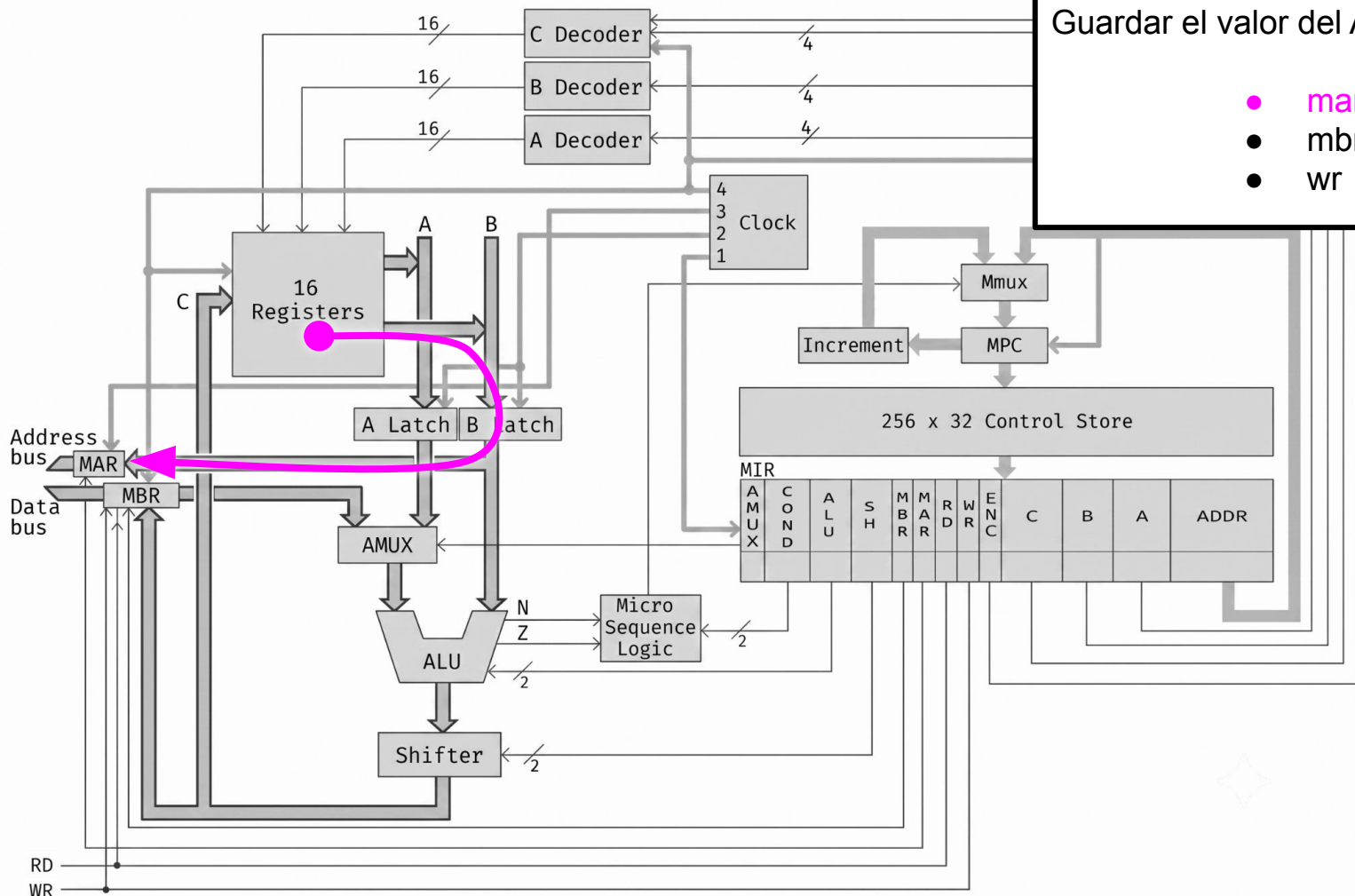
Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxx	LODD	Load direct	$ac := m[x]$
0001xxxxxxxxxx	STOD	Store direct	$m[x] := ac$
0010xxxxxxxxxx	ADDD	Add direct	$ac := ac + m[x]$
0011xxxxxxxxxx	SUBD	Subtract direct	$ac := ac - m[x]$
0100xxxxxxxxxx	JPOS	Jump positive	if $ac \geq 0$ then $pc := x$

STORE DIRECT

...
 STOD 9
 ...

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr



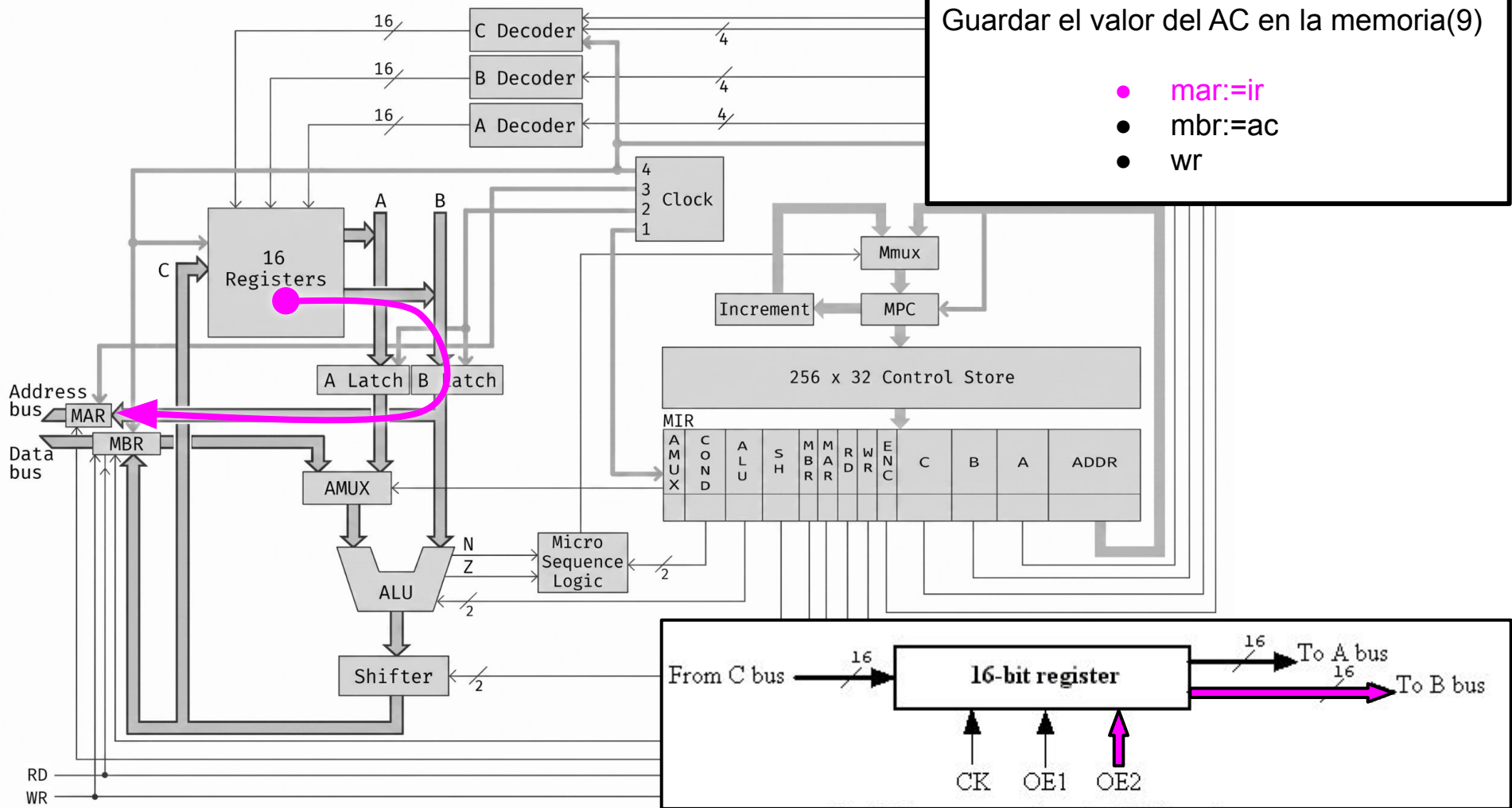
Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxx	LODD	Load direct	$ac := m[x]$
0001xxxxxxxxxx	STOD	Store direct	$m[x] := ac$
0010xxxxxxxxxx	ADDD	Add direct	$ac := ac + m[x]$
0011xxxxxxxxxx	SUBD	Subtract direct	$ac := ac - m[x]$
0100xxxxxxxxxx	JPOS	Jump positive	if $ac \geq 0$ then $pc := x$

STORE DIRECT

...
STOD 9
...

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr



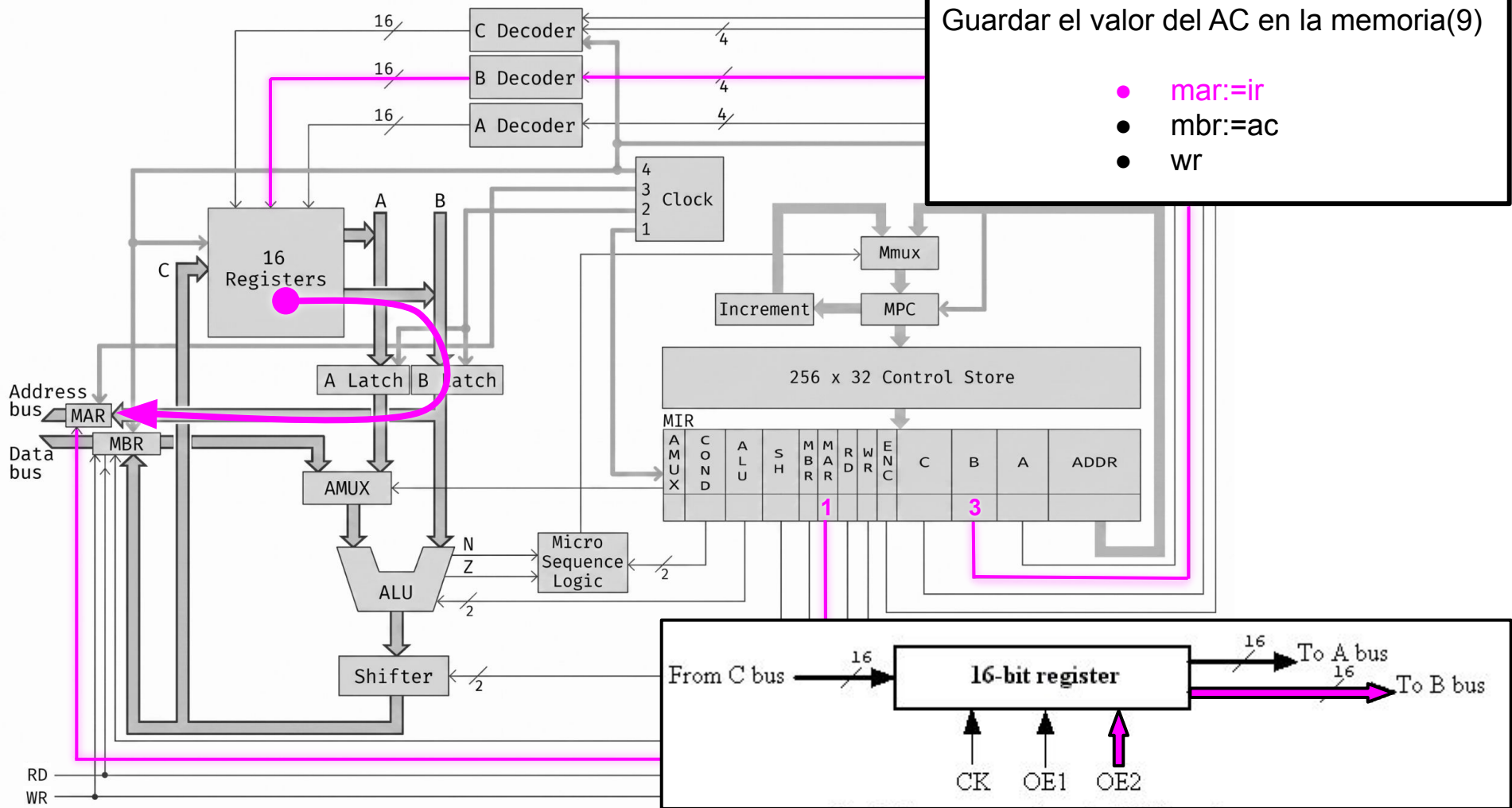
Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxx	LODD	Load direct	$ac := m[x]$
0001xxxxxxxxxx	STOD	Store direct	$m[x] := ac$
0010xxxxxxxxxx	ADDD	Add direct	$ac := ac + m[x]$
0011xxxxxxxxxx	SUBD	Subtract direct	$ac := ac - m[x]$
0100xxxxxxxxxx	JPOS	Jump positive	if $ac \geq 0$ then $pc := x$

STORE DIRECT

...
STOD 9
...

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr



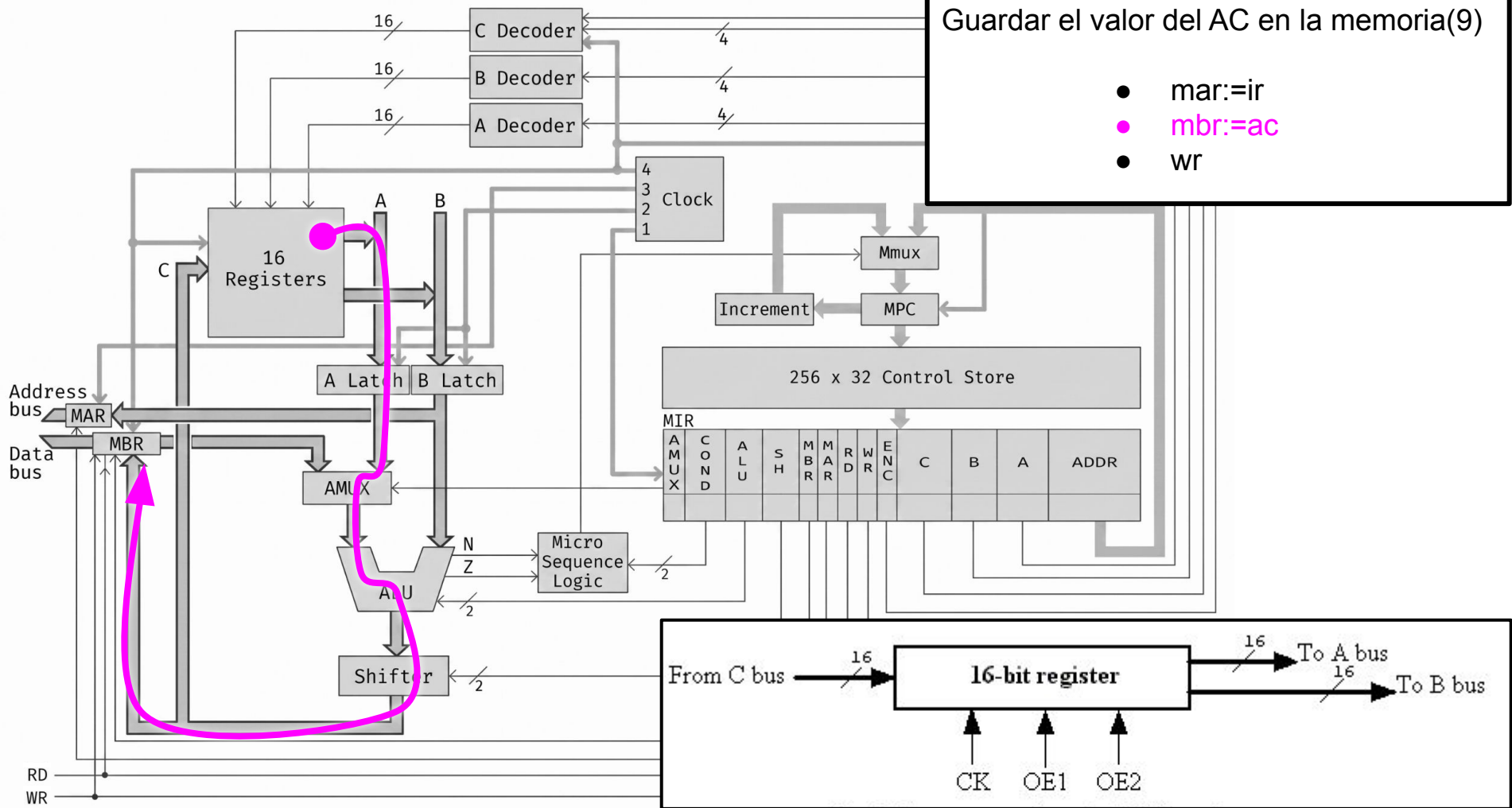
Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxx	LODD	Load direct	$ac := m[x]$
0001xxxxxxxxxx	STOD	Store direct	$m[x] := ac$
0010xxxxxxxxxx	ADDD	Add direct	$ac := ac + m[x]$
0011xxxxxxxxxx	SUBD	Subtract direct	$ac := ac - m[x]$
0100xxxxxxxxxx	JPOS	Jump positive	if $ac \geq 0$ then $pc := x$

STORE DIRECT

...
STOD 9
...

Guardar el valor del AC en la memoria(9)

- $mar := ir$
- $mbr := ac$
- wr



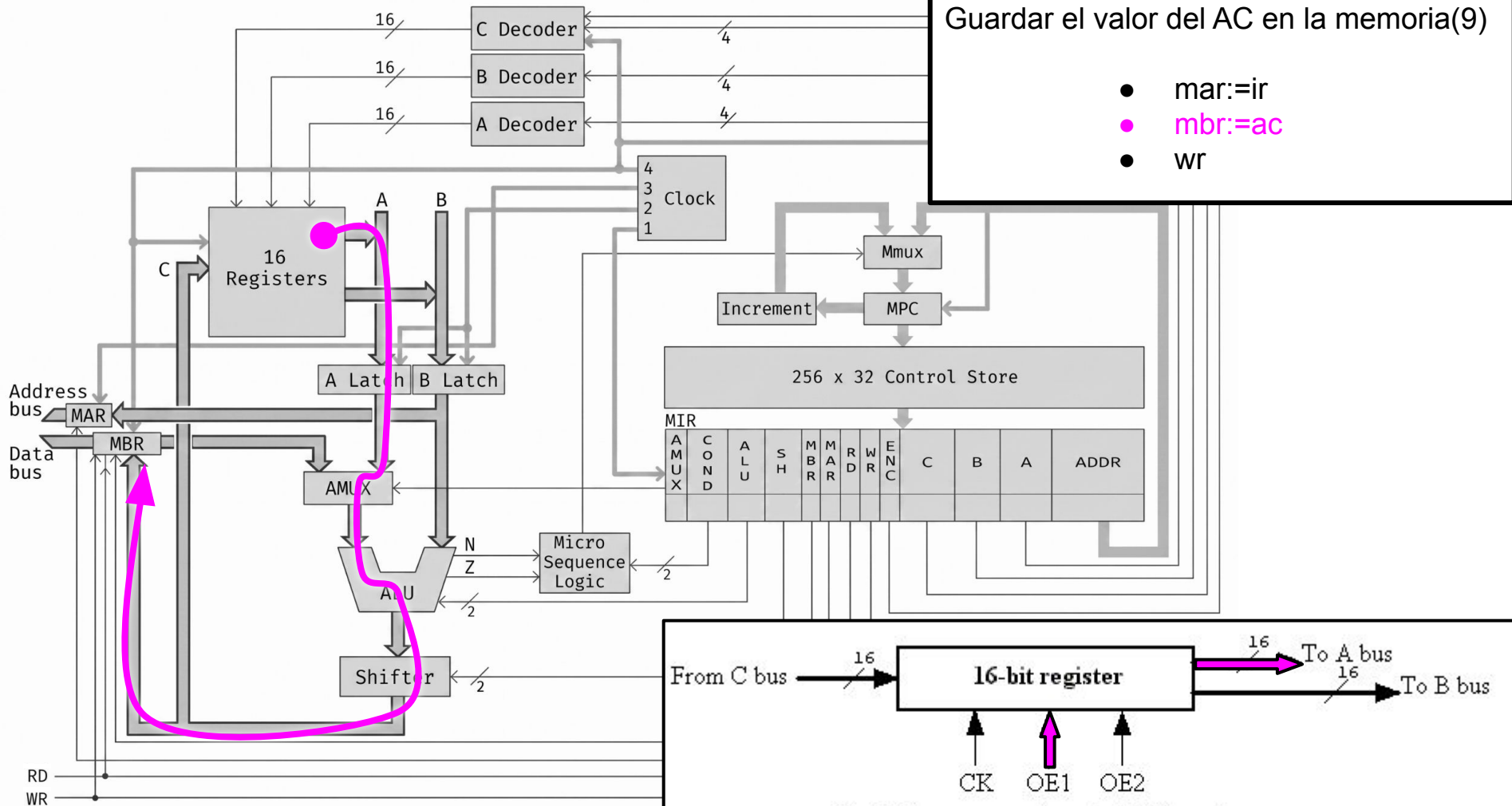
Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxx	LODD	Load direct	$ac := m[x]$
0001xxxxxxxxxx	STOD	Store direct	$m[x] := ac$
0010xxxxxxxxxx	ADDD	Add direct	$ac := ac + m[x]$
0011xxxxxxxxxx	SUBD	Subtract direct	$ac := ac - m[x]$
0100xxxxxxxxxx	JPOS	Jump positive	if $ac \geq 0$ then $pc := x$

STORE DIRECT

...
 STOD 9
 ...

Guardar el valor del AC en la memoria(9)

- $mar := ir$
- $mbr := ac$
- wr



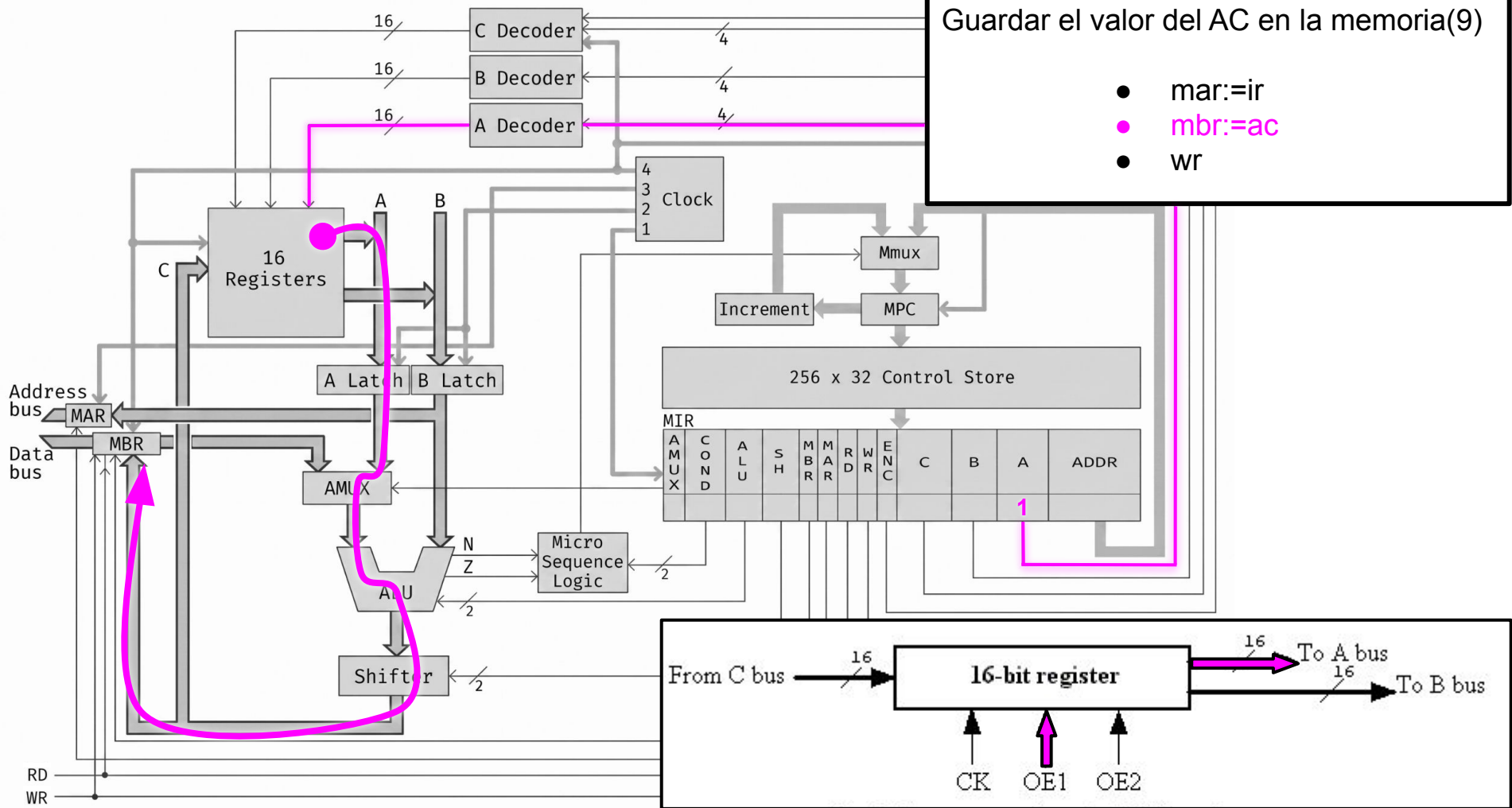
Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxx	LODD	Load direct	$ac := m[x]$
0001xxxxxxxxxx	STOD	Store direct	$m[x] := ac$
0010xxxxxxxxxx	ADDD	Add direct	$ac := ac + m[x]$
0011xxxxxxxxxx	SUBD	Subtract direct	$ac := ac - m[x]$
0100xxxxxxxxxx	JPOS	Jump positive	if $ac \geq 0$ then $pc := x$

STORE DIRECT

...
STOD 9
...

Guardar el valor del AC en la memoria(9)

- $mar := ir$
- $mbr := ac$
- wr



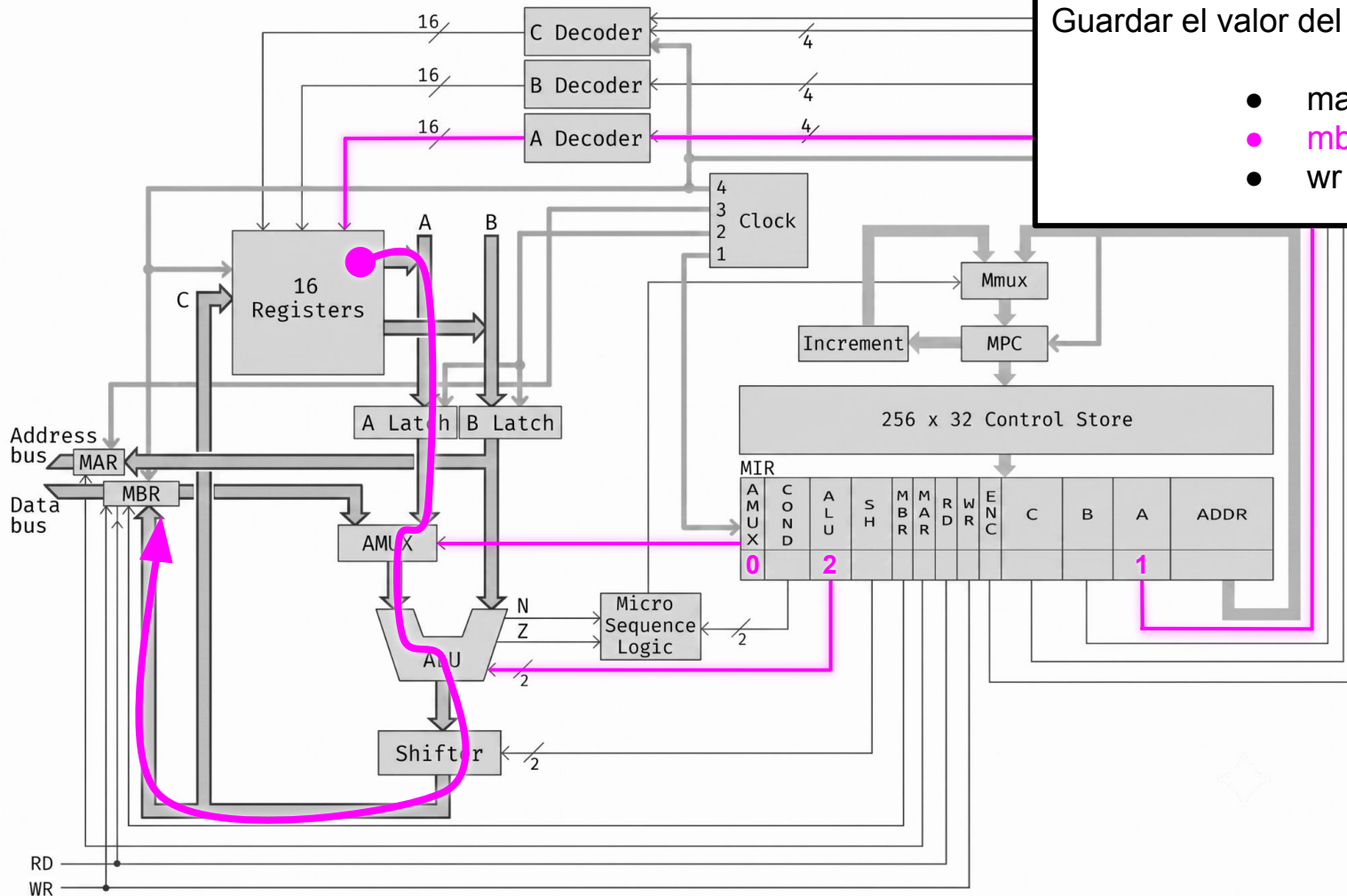
Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxx	LODD	Load direct	$ac := m[x]$
0001xxxxxxxxxx	STOD	Store direct	$m[x] := ac$
0010xxxxxxxxxx	ADDD	Add direct	$ac := ac + m[x]$
0011xxxxxxxxxx	SUBD	Subtract direct	$ac := ac - m[x]$
0100xxxxxxxxxx	JPOS	Jump positive	if $ac \geq 0$ then $pc := x$

STORE DIRECT

...
 STOD 9
 ...

Guardar el valor del AC en la memoria(9)

- $mar := ir$
- $mbr := ac$
- wr



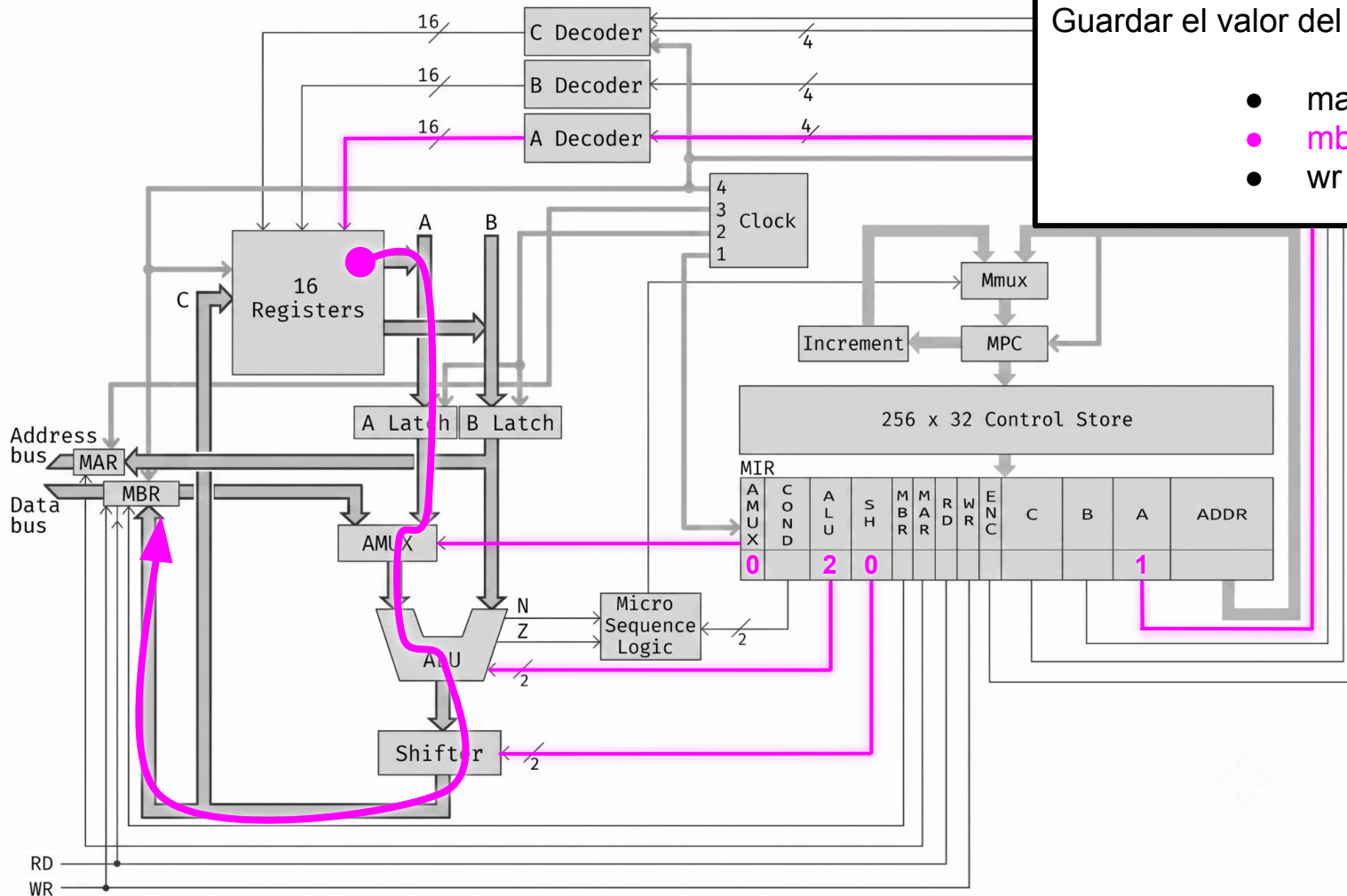
Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	$ac := m[x]$
0001xxxxxxxxxxxx	STOD	Store direct	$m[x] := ac$
0010xxxxxxxxxxxx	ADDD	Add direct	$ac := ac + m[x]$
0011xxxxxxxxxxxx	SUBD	Subtract direct	$ac := ac - m[x]$
0100xxxxxxxxxxxx	JPOS	Jump positive	if $ac \geq 0$ then $pc := x$

STORE DIRECT

...
STOD 9
...

Guardar el valor del AC en la memoria(9)

- $mar := ir$
- $mbr := ac$
- wr



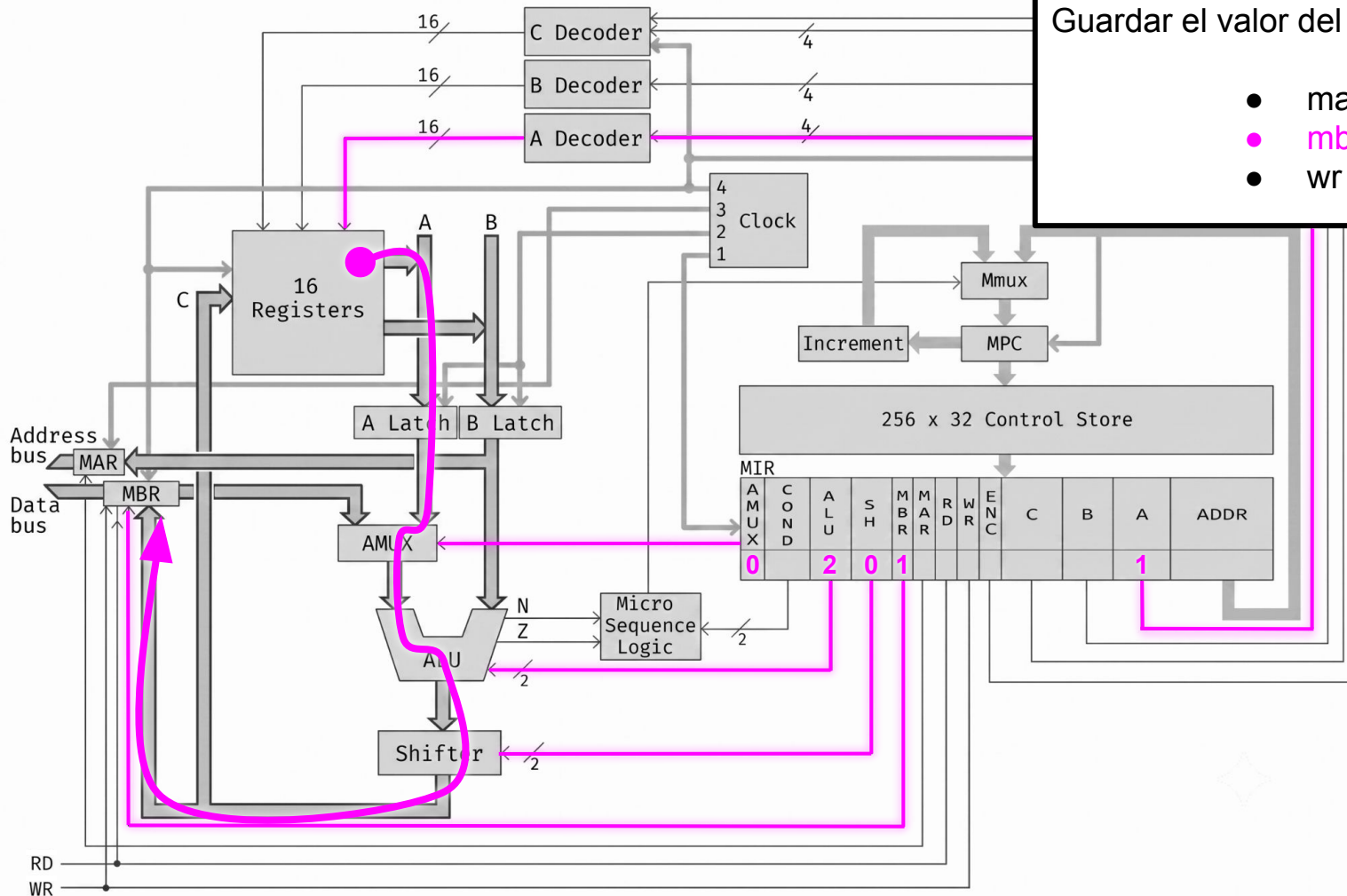
Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxx	LODD	Load direct	$ac := m[x]$
0001xxxxxxxxxx	STOD	Store direct	$m[x] := ac$
0010xxxxxxxxxx	ADDD	Add direct	$ac := ac + m[x]$
0011xxxxxxxxxx	SUBD	Subtract direct	$ac := ac - m[x]$
0100xxxxxxxxxx	JPOS	Jump positive	if $ac \geq 0$ then $pc := x$

STORE DIRECT

...
 STOD 9
 ...

Guardar el valor del AC en la memoria(9)

- $mar := ir$
- $mbr := ac$
- wr



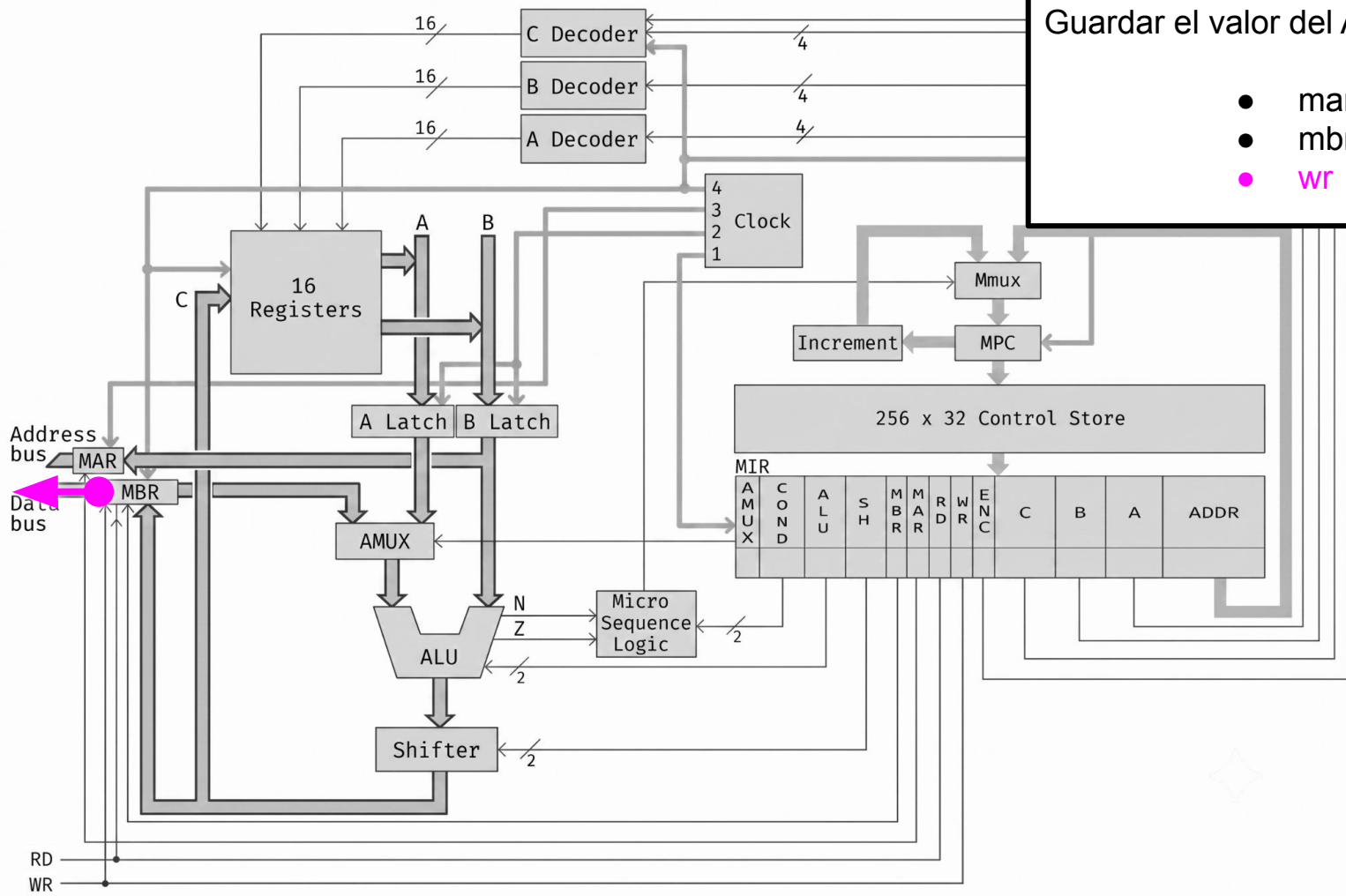
Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxx	LODD	Load direct	$ac := m[x]$
0001xxxxxxxxxx	STOD	Store direct	$m[x] := ac$
0010xxxxxxxxxx	ADDD	Add direct	$ac := ac + m[x]$
0011xxxxxxxxxx	SUBD	Subtract direct	$ac := ac - m[x]$
0100xxxxxxxxxx	JPOS	Jump positive	if $ac \geq 0$ then $pc := x$

STORE DIRECT

...
STOD 9
...

Guardar el valor del AC en la memoria(9)

- $mar := ir$
- $mbr := ac$
- WR



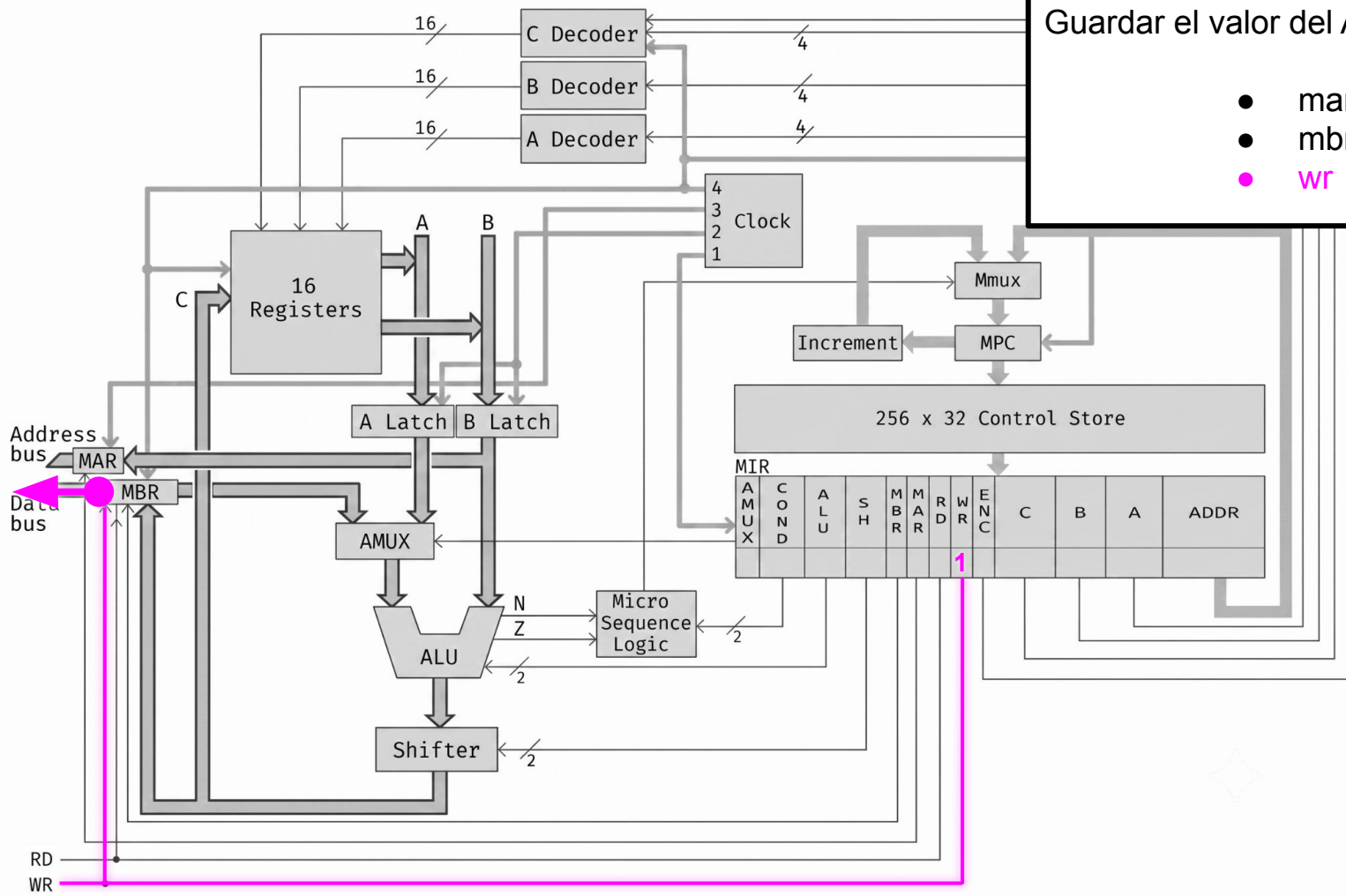
Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxx	LODD	Load direct	$ac := m[x]$
0001xxxxxxxxxx	STOD	Store direct	$m[x] := ac$
0010xxxxxxxxxx	ADDD	Add direct	$ac := ac + m[x]$
0011xxxxxxxxxx	SUBD	Subtract direct	$ac := ac - m[x]$
0100xxxxxxxxxx	JPOS	Jump positive	if $ac \geq 0$ then $pc := x$

STORE DIRECT

...
 STOD 9
 ...

Guardar el valor del AC en la memoria(9)

- $mar := ir$
- $mbr := ac$
- WR



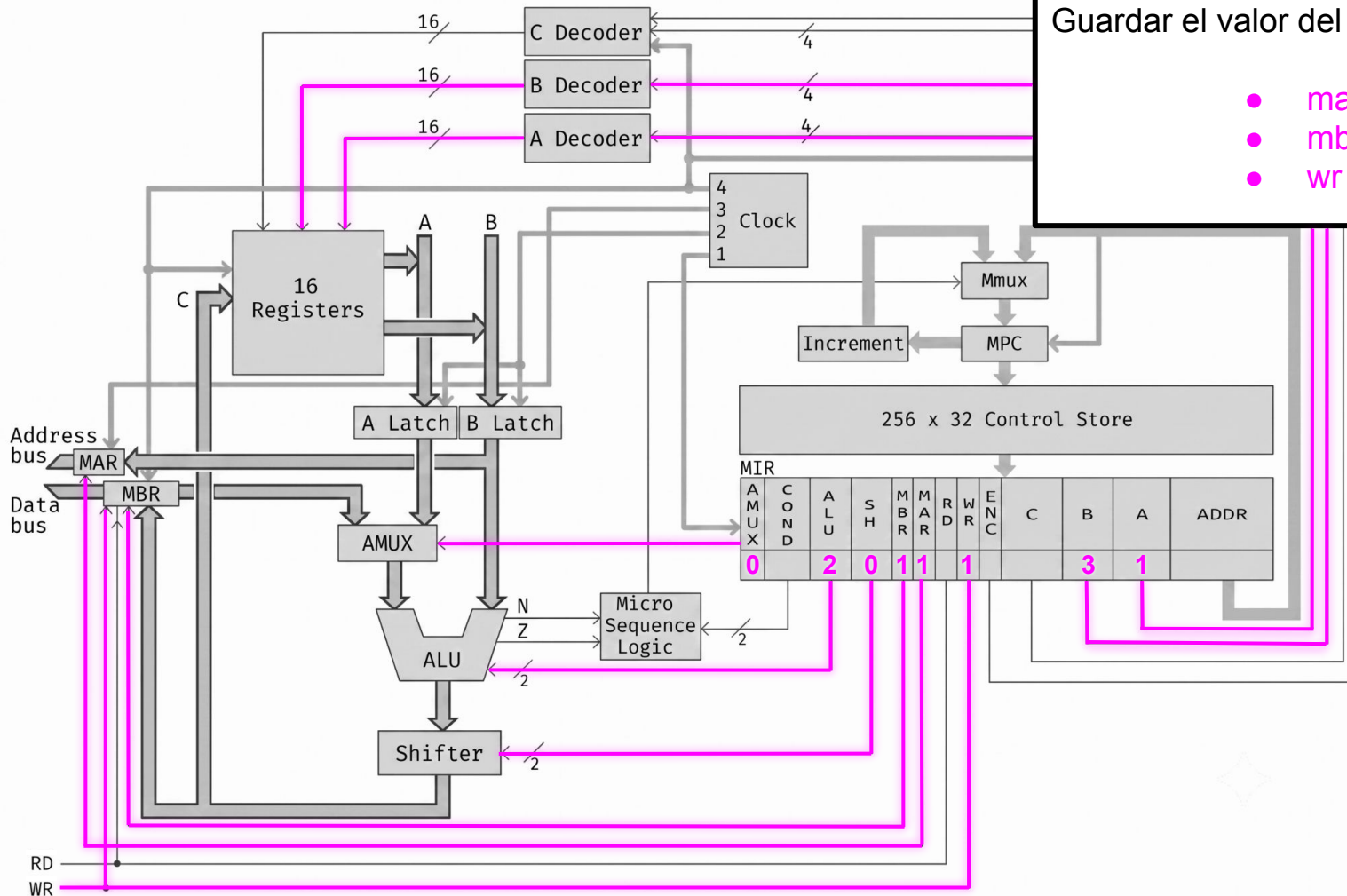
Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxx	LODD	Load direct	$ac := m[x]$
0001xxxxxxxxxx	STOD	Store direct	$m[x] := ac$
0010xxxxxxxxxx	ADDD	Add direct	$ac := ac + m[x]$
0011xxxxxxxxxx	SUBD	Subtract direct	$ac := ac - m[x]$
0100xxxxxxxxxx	JPOS	Jump positive	if $ac \geq 0$ then $pc := x$

STORE DIRECT

...
 STOD 9
 ...

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr



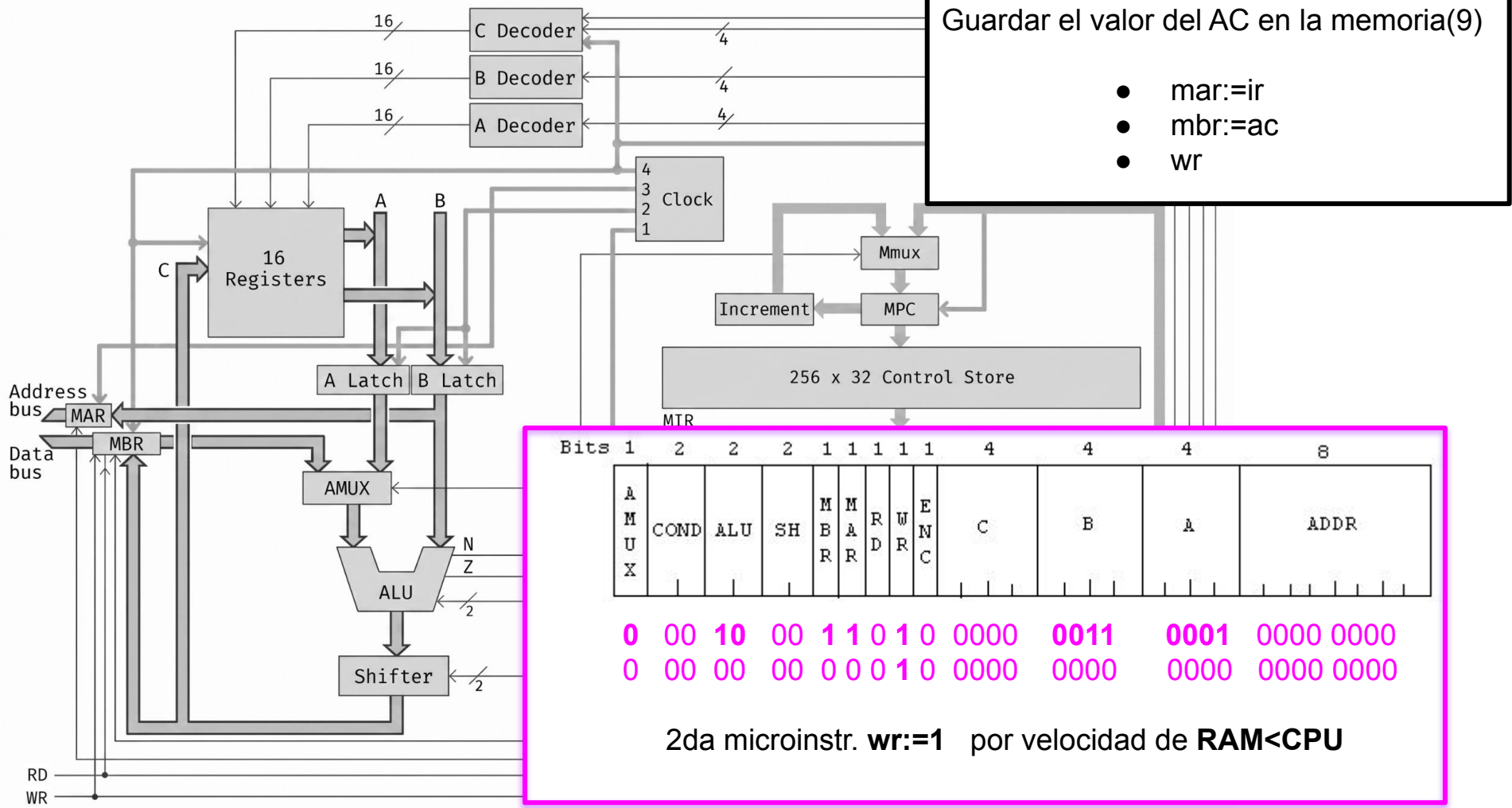
Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxx	LODD	Load direct	$ac := m[x]$
0001xxxxxxxxxx	STOD	Store direct	$m[x] := ac$
0010xxxxxxxxxx	ADDD	Add direct	$ac := ac + m[x]$
0011xxxxxxxxxx	SUBD	Subtract direct	$ac := ac - m[x]$
0100xxxxxxxxxx	JPOS	Jump positive	if $ac \geq 0$ then $pc := x$

STORE DIRECT

...
STOD 9
...

Guardar el valor del AC en la memoria(9)

- $mar := ir$
- $mbr := ac$
- wr



2da microinstr. $wr:=1$ por velocidad de RAM<CPU

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	$ac := m[x]$
0001xxxxxxxxxxxx	STOD	Store direct	$m[x] := ac$
0010xxxxxxxxxxxx	ADDD	Add direct	$ac := ac + m[x]$
0011xxxxxxxxxxxx	SUBD	Subtract direct	$ac := ac - m[x]$
0100xxxxxxxxxxxx	JPOS	Jump positive	if $ac \geq 0$ then $pc := x$
0101xxxxxxxxxxxx	JZER	Jump zero	if $ac = 0$ then $pc := x$
0110xxxxxxxxxxxx	JUMP	Jump	$pc := x$
0111xxxxxxxxxxxx	LOCO	Load constant	$ac := x$ ($0 \leq x \leq 4095$)
1000xxxxxxxxxxxx	LODL	Load local	$ac := m[sp+x]$
1001xxxxxxxxxxxx	STOL	Store local	$m[x+sp] := ac$
1010xxxxxxxxxxxx	ADDL	Add local	$ac := ac + m[sp+x]$
1011xxxxxxxxxxxx	SUBL	Subtract local	$ac := ac - m[sp+x]$
1100xxxxxxxxxxxx	JNEG	Jump negative	if $ac < 0$ then $pc := x$
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if $ac \neq 0$ then $pc := x$
1110xxxxxxxxxxxx	CALL	Call procedure	$sp := sp - 1$; $m[sp] := pc$; $pc := x$
1111000000000000	PSHI	Push indirect	$sp := sp - 1$; $m[sp] := m[ac]$
1111001000000000	POPI	Pop indirect	$m[ac] := m[sp]$; $sp := sp + 1$
1111010000000000	PUSH	Push onto stack	$sp := sp - 1$; $m[sp] := ac$
1111011000000000	POP	Pop from stack	$ac := m[sp]$; $sp := sp + 1$
1111100000000000	RETN	Return	$pc := m[sp]$; $sp := sp + 1$
1111101000000000	SWAP	Swap ac sp	$tmp := ac$; $ac := sp$; $sp := tmp$
11111100yyyyyyyy	INSP	Increment sp	$sp := sp + y$ ($0 \leq y \leq 255$)
11111110yyyyyyyy	DESP	Decrement sp	$sp := sp - y$ ($0 \leq y \leq 255$)

Pila o Stack

- La pila se ubica en la parte más alta de la memoria
- SP apunta siempre al tope de la pila
- POP saca el valor del tope de la pila
- PUSH pone un valor en la pila

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxxx	JPOS	Jump positive	if ac>=0 then pc:=x
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxxxxxxxxxx	LOCO	Load constant	ac:=x (0<=x<=4095)
1000xxxxxxxxxxxx	LODL	Load local	ac:=m[sp+x]
1001xxxxxxxxxxxx	STOL	Store local	m[x+sp]:=ac
1010xxxxxxxxxxxx	ADDL	Add local	ac:=ac+m[sp+x]
1011xxxxxxxxxxxx	SUBL	Subtract local	ac:=ac-m[sp+x]
1100xxxxxxxxxxxx	JNEG	Jump negative	if ac<0 then pc:=x
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if ac<>0 then pc:=x
1110xxxxxxxxxxxx	CALL	Call procedure	sp:=sp-1; m[sp]:=pc; pc:=x
1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac
1111011000000000	POP	Pop from stack	ac:=m[sp]; sp:=sp+ 1
1111100000000000	RETN	Return	pc:=m[sp]; sp:=sp+ 1
1111101000000000	SWAP	Swap ac sp	tmp:=ac; ac:=sp; sp:=tmp
11111100yyyyyyyy	INSP	Increment sp	sp:=sp+y (0<=y<=255)
11111110yyyyyyyy	DESP	Decrement sp	sp:=sp-y (0<=y<=255)

Pila o Stack

- La pila se ubica en la parte más alta de la memoria
- SP apunta siempre al tope de la pila
- POP saca el valor del tope de la pila
- PUSH pone un valor en la pila

SP→	0xFFD	7
	0xFFE	5
	0xFFF	101

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	$ac := m[x]$
0001xxxxxxxxxxxx	STOD	Store direct	$m[x] := ac$
0010xxxxxxxxxxxx	ADDD	Add direct	$ac := ac + m[x]$
0011xxxxxxxxxxxx	SUBD	Subtract direct	$ac := ac - m[x]$
0100xxxxxxxxxxxx	JPOS	Jump positive	if $ac \geq 0$ then $pc := x$
0101xxxxxxxxxxxx	JZER	Jump zero	if $ac = 0$ then $pc := x$
0110xxxxxxxxxxxx	JUMP	Jump	$pc := x$
0111xxxxxxxxxxxx	LOCO	Load constant	$ac := x$ ($0 \leq x \leq 4095$)
1000xxxxxxxxxxxx	LODL	Load local	$ac := m[sp+x]$
1001xxxxxxxxxxxx	STOL	Store local	$m[x+sp] := ac$
1010xxxxxxxxxxxx	ADDL	Add local	$ac := ac + m[sp+x]$
1011xxxxxxxxxxxx	SUBL	Subtract local	$ac := ac - m[sp+x]$
1100xxxxxxxxxxxx	JNEG	Jump negative	if $ac < 0$ then $pc := x$
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if $ac \neq 0$ then $pc := x$
1110xxxxxxxxxxxx	CALL	Call procedure	$sp := sp - 1$; $m[sp] := pc$; $pc := x$
1111000000000000	PSHI	Push indirect	$sp := sp - 1$; $m[sp] := m[ac]$
1111001000000000	POPI	Pop indirect	$m[ac] := m[sp]$; $sp := sp + 1$
1111010000000000	PUSH	Push onto stack	$sp := sp - 1$; $m[sp] := ac$
1111011000000000	POP	Pop from stack	$ac := m[sp]$; $sp := sp + 1$
1111100000000000	RETN	Return	$pc := m[sp]$; $sp := sp + 1$
1111101000000000	SWAP	Swap ac sp	$tmp := ac$; $ac := sp$; $sp := tmp$
11111100yyyyyyyy	INSP	Increment sp	$sp := sp + y$ ($0 \leq y \leq 255$)
11111110yyyyyyyy	DESP	Decrement sp	$sp := sp - y$ ($0 \leq y \leq 255$)

POP

- $SP \leftrightarrow 0xFFD$
- $m[sp] \leftrightarrow$ valor en el tope de la pila

SP→	0xFFD	7
	0xFFE	5
	0xFFF	101

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	$ac := m[x]$
0001xxxxxxxxxxxx	STOD	Store direct	$m[x] := ac$
0010xxxxxxxxxxxx	ADDD	Add direct	$ac := ac + m[x]$
0011xxxxxxxxxxxx	SUBD	Subtract direct	$ac := ac - m[x]$
0100xxxxxxxxxxxx	JPOS	Jump positive	if $ac \geq 0$ then $pc := x$
0101xxxxxxxxxxxx	JZER	Jump zero	if $ac = 0$ then $pc := x$
0110xxxxxxxxxxxx	JUMP	Jump	$pc := x$
0111xxxxxxxxxxxx	LOCO	Load constant	$ac := x$ ($0 \leq x \leq 4095$)
1000xxxxxxxxxxxx	LODL	Load local	$ac := m[sp+x]$
1001xxxxxxxxxxxx	STOL	Store local	$m[x+sp] := ac$
1010xxxxxxxxxxxx	ADDL	Add local	$ac := ac + m[sp+x]$
1011xxxxxxxxxxxx	SUBL	Subtract local	$ac := ac - m[sp+x]$
1100xxxxxxxxxxxx	JNEG	Jump negative	if $ac < 0$ then $pc := x$
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if $ac \neq 0$ then $pc := x$
1110xxxxxxxxxxxx	CALL	Call procedure	$sp := sp - 1$; $m[sp] := pc$; $pc := x$
1111000000000000	PSHI	Push indirect	$sp := sp - 1$; $m[sp] := m[ac]$
1111001000000000	POPI	Pop indirect	$m[ac] := m[sp]$; $sp := sp + 1$
1111010000000000	PUSH	Push onto stack	$sp := sp - 1$; $m[sp] := ac$
1111011000000000	POP	Pop from stack	$ac := m[sp]$; $sp := sp + 1$
1111100000000000	RETN	Return	$pc := m[sp]$; $sp := sp + 1$
1111101000000000	SWAP	Swap ac sp	$tmp := ac$; $ac := sp$; $sp := tmp$
11111100yyyyyyyy	INSP	Increment sp	$sp := sp + y$ ($0 \leq y \leq 255$)
11111110yyyyyyyy	DESP	Decrement sp	$sp := sp - y$ ($0 \leq y \leq 255$)

POP

- $SP \leftrightarrow 0xFFD$
- $m[sp] \leftrightarrow$ valor en el tope de la pila
- $SP := SP + 1 \leftrightarrow$ muevo el puntero

SP→	0xFFD	7
	0xFFE	5
	0xFFF	101

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxxx	JPOS	Jump positive	if ac>=0 then pc:=x
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxxxxxxxxxx	LOCO	Load constant	ac:=x (0<=x<=4095)
1000xxxxxxxxxxxx	LODL	Load local	ac:=m[sp+x]
1001xxxxxxxxxxxx	STOL	Store local	m[x+sp]:=ac
1010xxxxxxxxxxxx	ADDL	Add local	ac:=ac+m[sp+x]
1011xxxxxxxxxxxx	SUBL	Subtract local	ac:=ac-m[sp+x]
1100xxxxxxxxxxxx	JNEG	Jump negative	if ac<0 then pc:=x
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if ac<>0 then pc:=x
1110xxxxxxxxxxxx	CALL	Call procedure	sp:=sp-1; m[sp]:=pc; pc:=x
1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac
1111011000000000	POP	Pop from stack	ac:=m[sp]; sp:=sp+ 1
1111100000000000	RETN	Return	pc:=m[sp]; sp:=sp+ 1
1111101000000000	SWAP	Swap ac sp	tmp:=ac; ac:=sp; sp:=tmp
11111100yyyyyyyy	INSP	Increment sp	sp:=sp+y (0<=y<=255)
11111110yyyyyyyy	DESP	Decrement sp	sp:=sp-y (0<=y<=255)

PUSH

- $SP \leftrightarrow 0xFFE$

SP→	0xFFD	7
	0xFFE	5
	0xFFF	101

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	$ac := m[x]$
0001xxxxxxxxxxxx	STOD	Store direct	$m[x] := ac$
0010xxxxxxxxxxxx	ADDD	Add direct	$ac := ac + m[x]$
0011xxxxxxxxxxxx	SUBD	Subtract direct	$ac := ac - m[x]$
0100xxxxxxxxxxxx	JPOS	Jump positive	if $ac \geq 0$ then $pc := x$
0101xxxxxxxxxxxx	JZER	Jump zero	if $ac = 0$ then $pc := x$
0110xxxxxxxxxxxx	JUMP	Jump	$pc := x$
0111xxxxxxxxxxxx	LOCO	Load constant	$ac := x$ ($0 \leq x \leq 4095$)
1000xxxxxxxxxxxx	LODL	Load local	$ac := m[sp+x]$
1001xxxxxxxxxxxx	STOL	Store local	$m[x+sp] := ac$
1010xxxxxxxxxxxx	ADDL	Add local	$ac := ac + m[sp+x]$
1011xxxxxxxxxxxx	SUBL	Subtract local	$ac := ac - m[sp+x]$
1100xxxxxxxxxxxx	JNEG	Jump negative	if $ac < 0$ then $pc := x$
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if $ac \neq 0$ then $pc := x$
1110xxxxxxxxxxxx	CALL	Call procedure	$sp := sp - 1$; $m[sp] := pc$; $pc := x$
1111000000000000	PSHI	Push indirect	$sp := sp - 1$; $m[sp] := m[ac]$
1111001000000000	POPI	Pop indirect	$m[ac] := m[sp]$; $sp := sp + 1$
1111010000000000	PUSH	Push onto stack	$sp := sp - 1$; $m[sp] := ac$
1111011000000000	POP	Pop from stack	$ac := m[sp]$; $sp := sp + 1$
1111100000000000	RETN	Return	$pc := m[sp]$; $sp := sp + 1$
1111101000000000	SWAP	Swap ac sp	$tmp := ac$; $ac := sp$; $sp := tmp$
11111100yyyyyyyy	INSP	Increment sp	$sp := sp + y$ ($0 \leq y \leq 255$)
11111110yyyyyyyy	DESP	Decrement sp	$sp := sp - y$ ($0 \leq y \leq 255$)

PUSH

- $SP \leftrightarrow 0xFFE$
- $SP := SP - 1 \leftrightarrow$ muevo el puntero
- $m[sp] \leftarrow$ nuevo valor en la pila

SP→	0xFFD	32
	0xFFE	5
	0xFFF	101

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxxx	JPOS	Jump positive	if ac>=0 then pc:=x
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxxxxxxxxxx	LOCO	Load constant	ac:=x (0<=x<=4095)
1000xxxxxxxxxxxx	LODL	Load local	ac:=m[sp+x]
1001xxxxxxxxxxxx	STOL	Store local	m[x+sp]:=ac
1010xxxxxxxxxxxx	ADDL	Add local	ac:=ac+m[sp+x]
1011xxxxxxxxxxxx	SUBL	Subtract local	ac:=ac-m[sp+x]
1100xxxxxxxxxxxx	JNEG	Jump negative	if ac<0 then pc:=x
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if ac<>0 then pc:=x
1110xxxxxxxxxxxx	CALL	Call procedure	sp:=sp-1; m[sp]:=pc; pc:=x
1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac
1111011000000000	POP	Pop from stack	ac:=m[sp]; sp:=sp+ 1
1111100000000000	RETN	Return	pc:=m[sp]; sp:=sp+ 1
1111101000000000	SWAP	Swap ac sp	tmp:=ac; ac:=sp; sp:=tmp
11111100yyyyyyyy	INSP	Increment sp	sp:=sp+y (0<=y<=255)
11111110yyyyyyyy	DESP	Decrement sp	sp:=sp-y (0<=y<=255)

POP INDIRECT

...
POPI
...

Saca del tope de la pila y lo guarda en la memoria(ac)

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	$ac := m[x]$
0001xxxxxxxxxxxx	STOD	Store direct	$m[x] := ac$
0010xxxxxxxxxxxx	ADDD	Add direct	$ac := ac + m[x]$
0011xxxxxxxxxxxx	SUBD	Subtract direct	$ac := ac - m[x]$
0100xxxxxxxxxxxx	JPOS	Jump positive	if $ac \geq 0$ then $pc := x$
0101xxxxxxxxxxxx	JZER	Jump zero	if $ac = 0$ then $pc := x$
0110xxxxxxxxxxxx	JUMP	Jump	$pc := x$
0111xxxxxxxxxxxx	LOCO	Load constant	$ac := x$ ($0 \leq x \leq 4095$)
1000xxxxxxxxxxxx	LODL	Load local	$ac := m[sp+x]$
1001xxxxxxxxxxxx	STOL	Store local	$m[x+sp] := ac$
1010xxxxxxxxxxxx	ADDL	Add local	$ac := ac + m[sp+x]$
1011xxxxxxxxxxxx	SUBL	Subtract local	$ac := ac - m[sp+x]$
1100xxxxxxxxxxxx	JNEG	Jump negative	if $ac < 0$ then $pc := x$
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if $ac \neq 0$ then $pc := x$
1110xxxxxxxxxxxx	CALL	Call procedure	$sp := sp - 1$; $m[sp] := pc$; $pc := x$
1111000000000000	PSHI	Push indirect	$sp := sp - 1$; $m[sp] := m[ac]$
1111001000000000	POPI	Pop indirect	$m[ac] := m[sp]$; $sp := sp + 1$
1111010000000000	PUSH	Push onto stack	$sp := sp - 1$; $m[sp] := ac$
1111011000000000	POP	Pop from stack	$ac := m[sp]$; $sp := sp + 1$
1111100000000000	RETN	Return	$pc := m[sp]$; $sp := sp + 1$
1111101000000000	SWAP	Swap ac sp	$tmp := ac$; $ac := sp$; $sp := tmp$
11111100yyyyyyyy	INSP	Increment sp	$sp := sp + y$ ($0 \leq y \leq 255$)
11111110yyyyyyyy	DESP	Decrement sp	$sp := sp - y$ ($0 \leq y \leq 255$)

POP INDIRECT

...
POPI
...

Saca del tope de la pila y lo guarda en la memoria(ac)

$m[ac] := m[sp]; \quad sp := sp + 1$

1111 0010 0000 0000

Binario	Mnem	Instrucción	Significado
0000xxxxxxxxxxxx	LODD	Load direct	$ac := m[x]$
0001xxxxxxxxxxxx	STOD	Store direct	$m[x] := ac$
0010xxxxxxxxxxxx	ADDD	Add direct	$ac := ac + m[x]$
0011xxxxxxxxxxxx	SUBD	Subtract direct	$ac := ac - m[x]$
0100xxxxxxxxxxxx	JPOS	Jump positive	if $ac \geq 0$ then $pc := x$
0101xxxxxxxxxxxx	JZER	Jump zero	if $ac = 0$ then $pc := x$
0110xxxxxxxxxxxx	JUMP	Jump	$pc := x$
0111xxxxxxxxxxxx	LOCO	Load constant	$ac := x$ ($0 \leq x \leq 4095$)
1000xxxxxxxxxxxx	LODL	Load local	$ac := m[sp+x]$
1001xxxxxxxxxxxx	STOL	Store local	$m[x+sp] := ac$
1010xxxxxxxxxxxx	ADDL	Add local	$ac := ac + m[sp+x]$
1011xxxxxxxxxxxx	SUBL	Subtract local	$ac := ac - m[sp+x]$
1100xxxxxxxxxxxx	JNEG	Jump negative	if $ac < 0$ then $pc := x$
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if $ac \neq 0$ then $pc := x$
1110xxxxxxxxxxxx	CALL	Call procedure	$sp := sp - 1$; $m[sp] := pc$; $pc := x$
1111000000000000	PSHI	Push indirect	$sp := sp - 1$; $m[sp] := m[ac]$
1111001000000000	POPI	Pop indirect	$m[ac] := m[sp]$; $sp := sp + 1$
1111010000000000	PUSH	Push onto stack	$sp := sp - 1$; $m[sp] := ac$
1111011000000000	POP	Pop from stack	$ac := m[sp]$; $sp := sp + 1$
1111100000000000	RETN	Return	$pc := m[sp]$; $sp := sp + 1$
1111101000000000	SWAP	Swap ac sp	$tmp := ac$; $ac := sp$; $sp := tmp$
11111100yyyyyyyy	INSP	Increment sp	$sp := sp + y$ ($0 \leq y \leq 255$)
11111110yyyyyyyy	DESP	Decrement sp	$sp := sp - y$ ($0 \leq y \leq 255$)

POP INDIRECT

...
POPI
...

Saca del tope de la pila y lo guarda en la memoria(ac)

$m[ac] := m[sp]$; $sp := sp + 1$

- $mar := sp$; rd
- $mar := ac$; wr
- $alu := sp + 1$
- $sp := alu$

Binario	Mnem	Instrucción	Significado
1111000000000000	PSHI	Push indirect	$sp := sp - 1; m[sp] := m[ac]$
1111001000000000	POPI	Pop indirect	$m[ac] := m[sp]; sp := sp + 1$
1111010000000000	PUSH	Push onto stack	$sp := sp - 1; m[sp] := ac$
1111011000000000	POP	Pop from stack	$ac := m[sp]; sp := sp + 1$
1111100000000000	RETN	Return	$pc := m[sp]; sp := sp + 1$

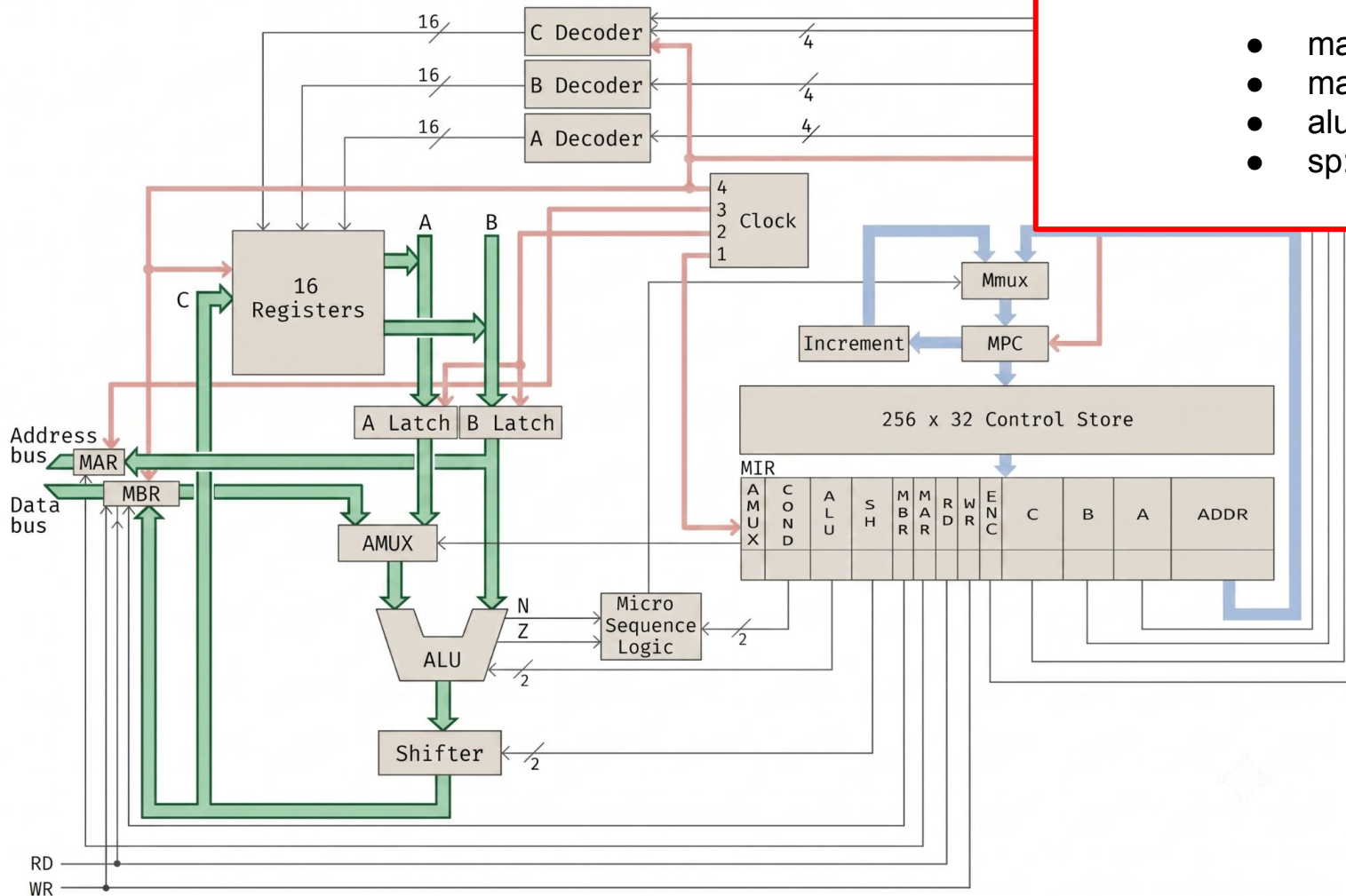
POP INDIRECT

POPI

Saca del tope de la pila y lo guarda en la memoria(ac)

$m[ac] := m[sp]; sp := sp + 1$

- $mar := sp; rd$
- $mar := ac; wr$
- $alu := sp + 1$
- $sp := alu$



Binario	Mnem	Instrucción	Significado
1111000000000000	PSHI	Push indirect	$sp := sp - 1; m[sp] := m[ac]$
1111001000000000	POPI	Pop indirect	$m[ac] := m[sp]; sp := sp + 1$
1111010000000000	PUSH	Push onto stack	$sp := sp - 1; m[sp] := ac$
1111011000000000	POP	Pop from stack	$ac := m[sp]; sp := sp + 1$
1111100000000000	RETN	Return	$pc := m[sp]; sp := sp + 1$

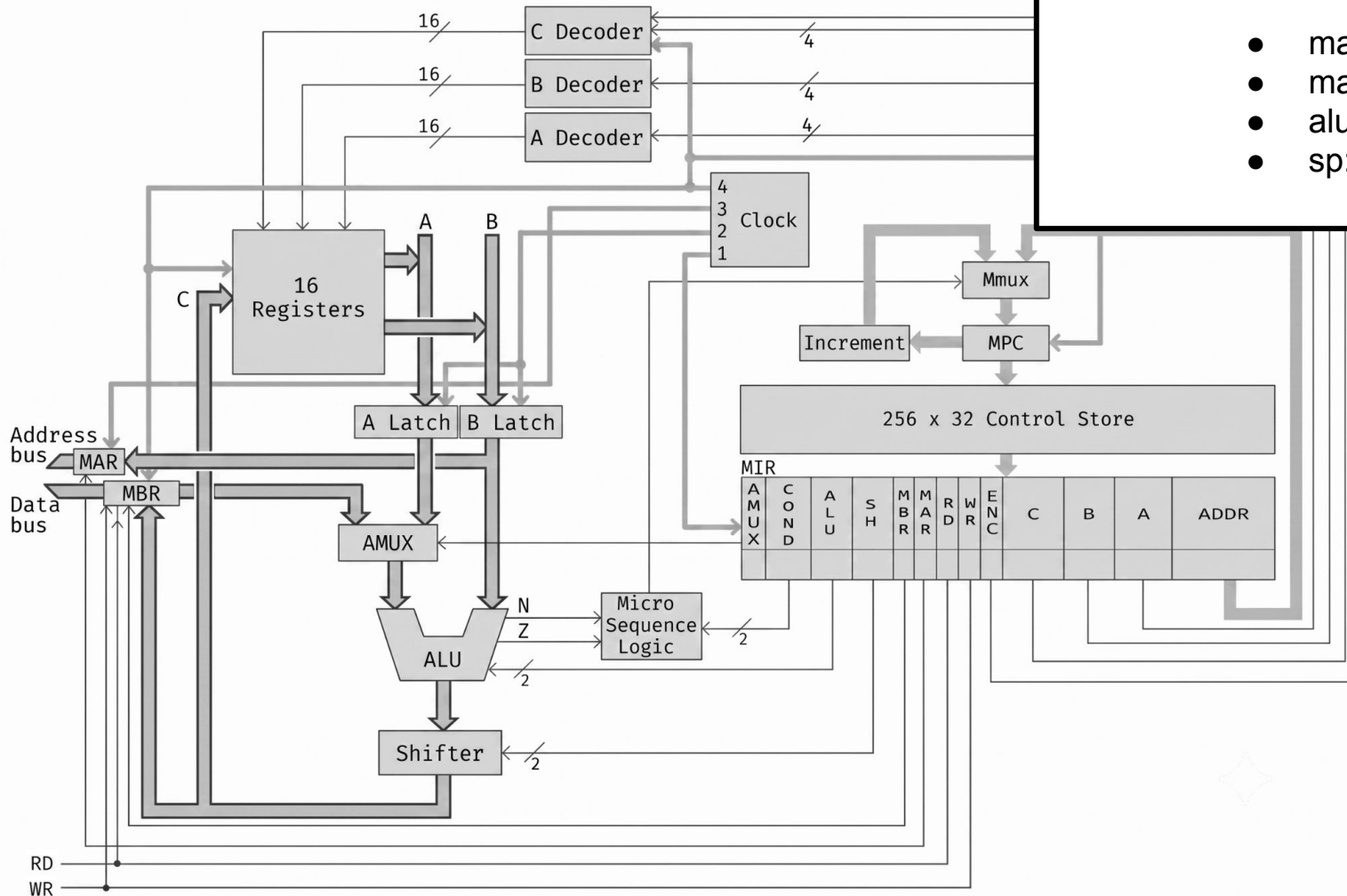
POP INDIRECT

POPI

Saca del tope de la pila y lo guarda en la memoria(ac)

$m[ac] := m[sp]; sp := sp + 1$

- $mar := sp; rd$
- $mar := ac; wr$
- $alu := sp + 1$
- $sp := alu$



Binario	Mnem	Instrucción	Significado
1111000000000000	PSHI	Push indirect	$sp := sp - 1; m[sp] := m[ac]$
1111001000000000	POPI	Pop indirect	$m[ac] := m[sp]; sp := sp + 1$
1111010000000000	PUSH	Push onto stack	$sp := sp - 1; m[sp] := ac$
1111011000000000	POP	Pop from stack	$ac := m[sp]; sp := sp + 1$
1111100000000000	RETN	Return	$pc := m[sp]; sp := sp + 1$

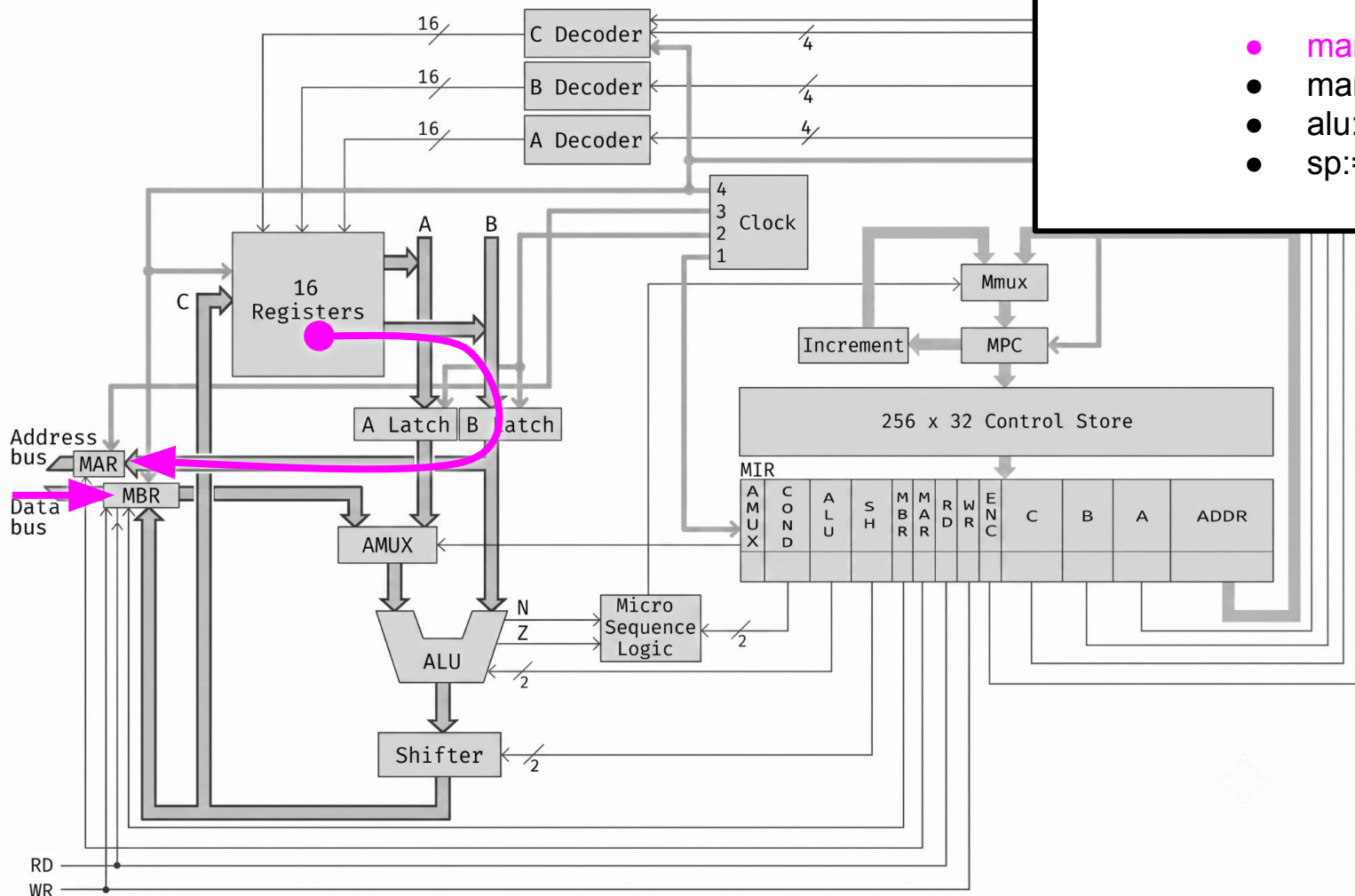
POP INDIRECT

POPI

Saca del tope de la pila y lo guarda en la memoria(ac)

$$m[ac] := m[sp]; \quad sp := sp + 1$$

- $mar := sp; rd$
- $mar := ac; wr$
- $alu := sp + 1$
- $sp := alu$



Binario	Mnem	Instrucción	Significado
1111000000000000	PSHI	Push indirect	$sp := sp - 1; m[sp] := m[ac]$
1111001000000000	POPI	Pop indirect	$m[ac] := m[sp]; sp := sp + 1$
1111010000000000	PUSH	Push onto stack	$sp := sp - 1; m[sp] := ac$
1111011000000000	POP	Pop from stack	$ac := m[sp]; sp := sp + 1$
1111100000000000	RETN	Return	$pc := m[sp]; sp := sp + 1$

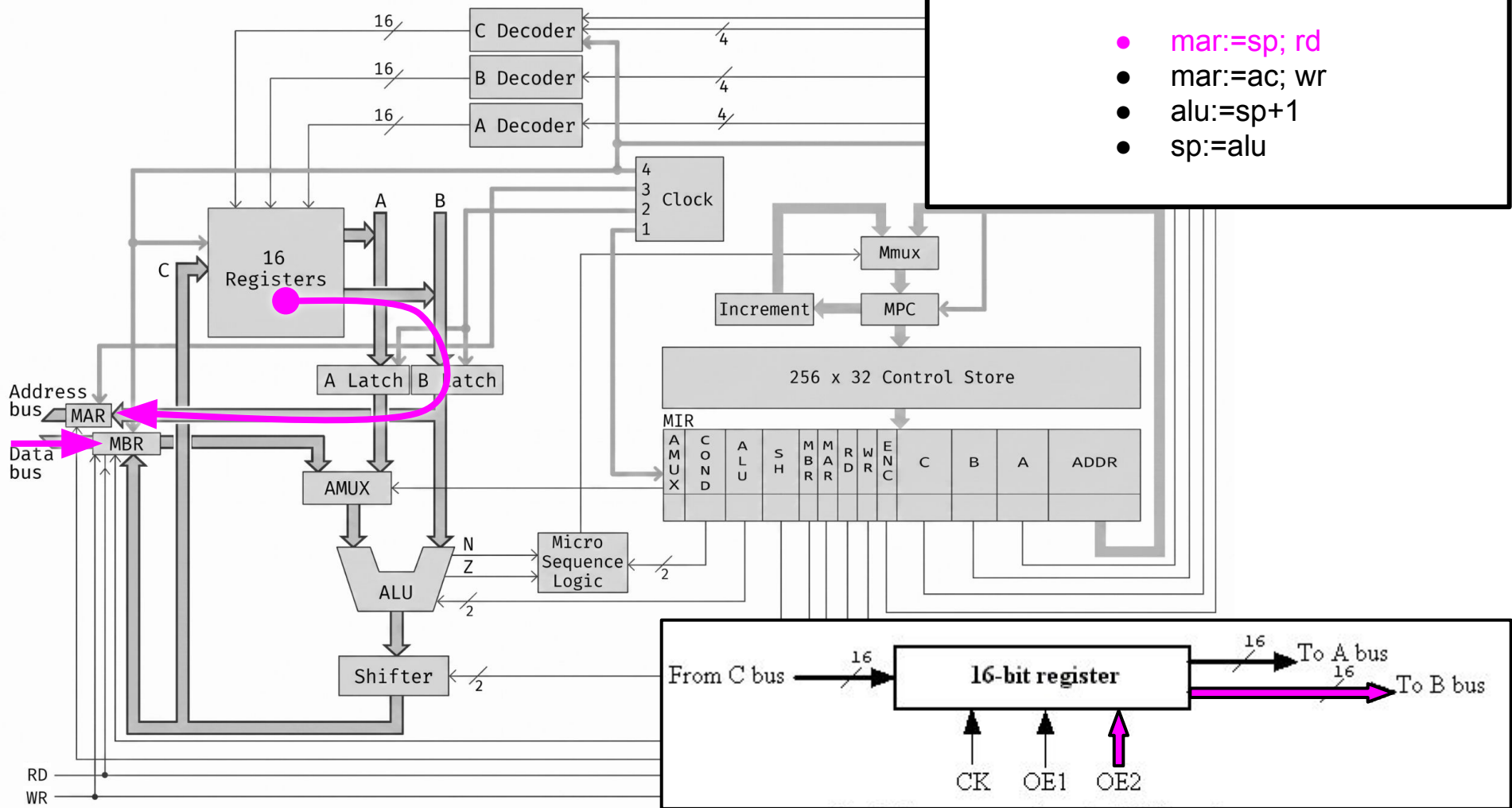
POP INDIRECT

POPI

Saca del tope de la pila y lo guarda en la memoria(ac)

$m[ac] := m[sp]; sp := sp + 1$

- $mar := sp; rd$
- $mar := ac; wr$
- $alu := sp + 1$
- $sp := alu$



Binario	Mnem	Instrucción	Significado
1111000000000000	PSHI	Push indirect	$sp := sp - 1; m[sp] := m[ac]$
1111001000000000	POPI	Pop indirect	$m[ac] := m[sp]; sp := sp + 1$
1111010000000000	PUSH	Push onto stack	$sp := sp - 1; m[sp] := ac$
1111011000000000	POP	Pop from stack	$ac := m[sp]; sp := sp + 1$
1111100000000000	RETN	Return	$pc := m[sp]; sp := sp + 1$

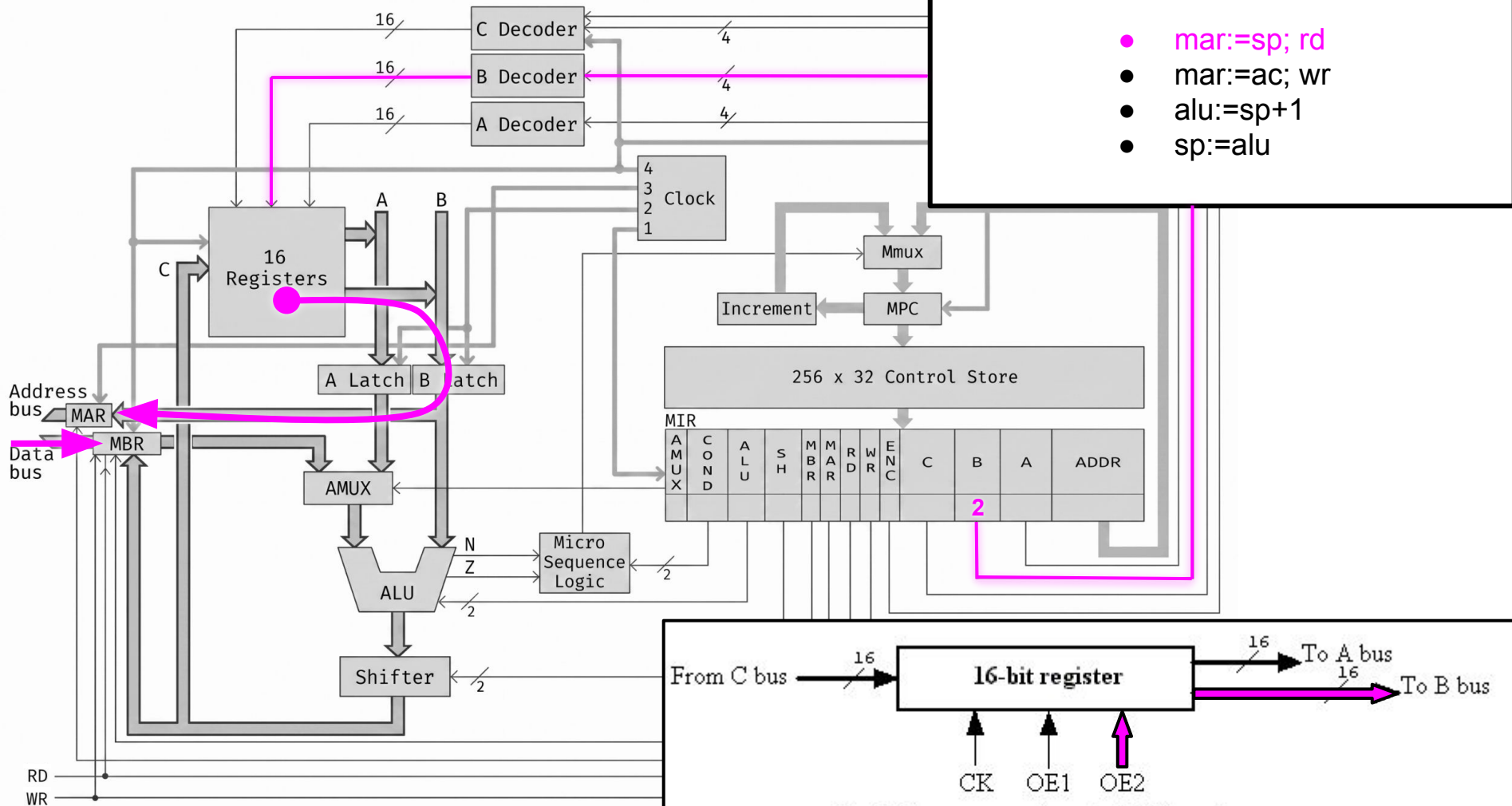
POP INDIRECT

POPI

Saca del tope de la pila y lo guarda en la memoria(ac)

$m[ac] := m[sp]; sp := sp + 1$

- $mar := sp; rd$
- $mar := ac; wr$
- $alu := sp + 1$
- $sp := alu$



Binario	Mnem	Instrucción	Significado
1111000000000000	PSHI	Push indirect	$sp := sp - 1; m[sp] := m[ac]$
1111001000000000	POPI	Pop indirect	$m[ac] := m[sp]; sp := sp + 1$
1111010000000000	PUSH	Push onto stack	$sp := sp - 1; m[sp] := ac$
1111011000000000	POP	Pop from stack	$ac := m[sp]; sp := sp + 1$
1111100000000000	RETN	Return	$pc := m[sp]; sp := sp + 1$

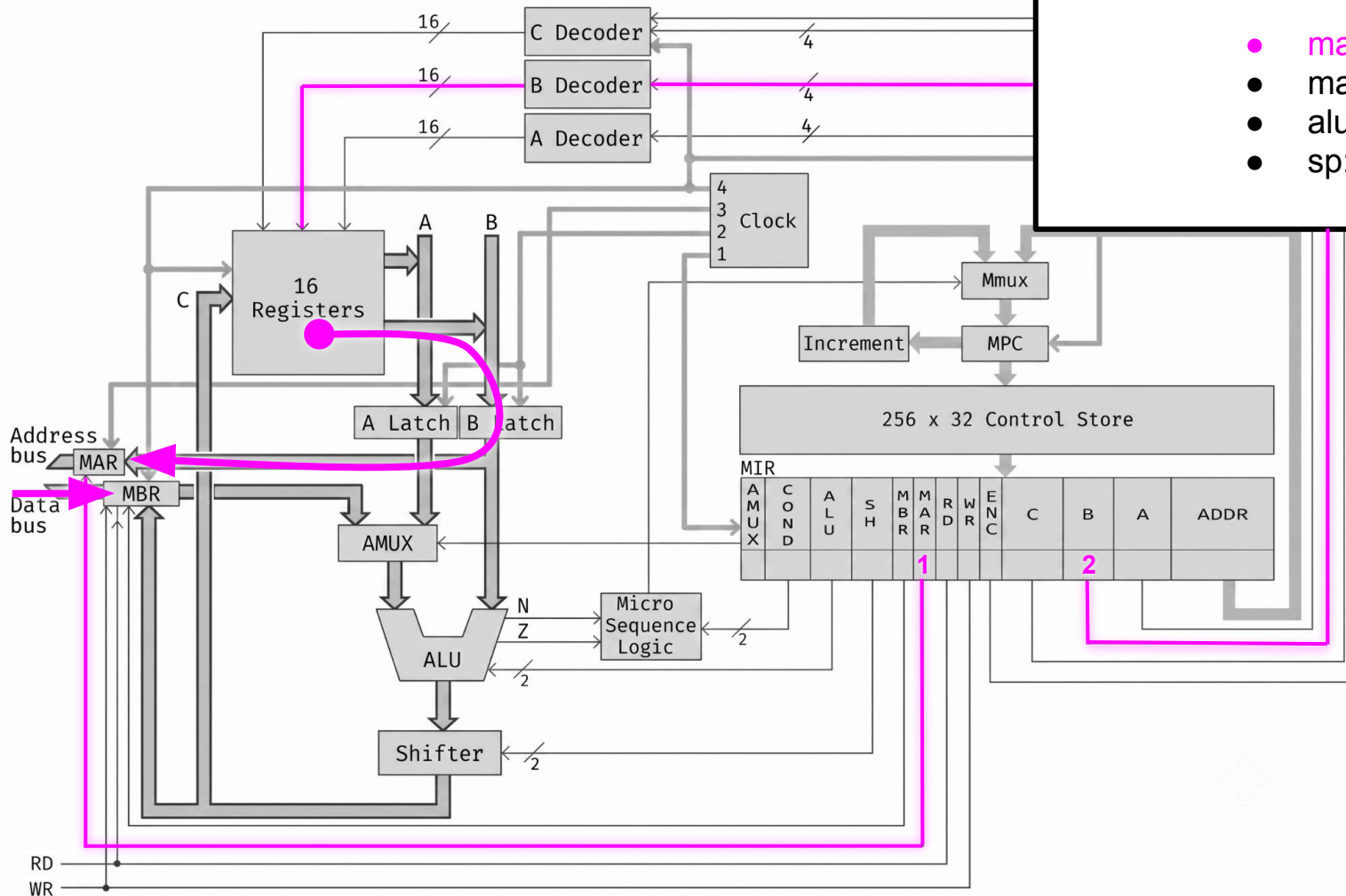
POP INDIRECT

POPI

Saca del tope de la pila y lo guarda en la memoria(ac)

$m[ac] := m[sp]; sp := sp + 1$

- $mar := sp; rd$
- $mar := ac; wr$
- $alu := sp + 1$
- $sp := alu$



Binario	Mnem	Instrucción	Significado
1111000000000000	PSHI	Push indirect	$sp := sp - 1; m[sp] := m[ac]$
1111001000000000	POPI	Pop indirect	$m[ac] := m[sp]; sp := sp + 1$
1111010000000000	PUSH	Push onto stack	$sp := sp - 1; m[sp] := ac$
1111011000000000	POP	Pop from stack	$ac := m[sp]; sp := sp + 1$
1111100000000000	RETN	Return	$pc := m[sp]; sp := sp + 1$

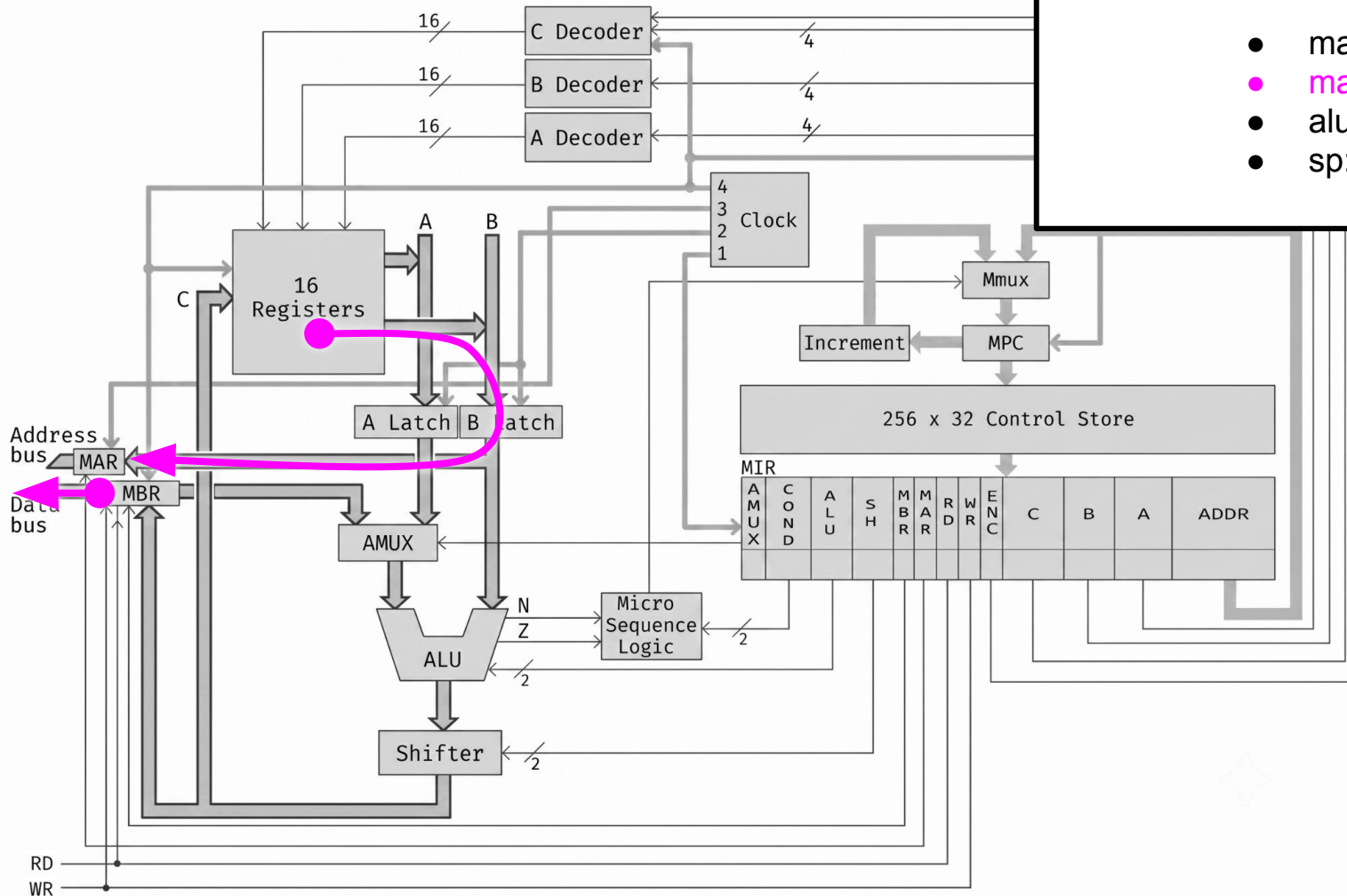
POP INDIRECT

POPI

Saca del tope de la pila y lo guarda en la memoria(ac)

$m[ac] := m[sp]; sp := sp + 1$

- $mar := sp; rd$
- $mar := ac; wr$
- $alu := sp + 1$
- $sp := alu$



Binario	Mnem	Instrucción	Significado
1111000000000000	PSHI	Push indirect	$sp := sp - 1; m[sp] := m[ac]$
1111001000000000	POPI	Pop indirect	$m[ac] := m[sp]; sp := sp + 1$
1111010000000000	PUSH	Push onto stack	$sp := sp - 1; m[sp] := ac$
1111011000000000	POP	Pop from stack	$ac := m[sp]; sp := sp + 1$
1111100000000000	RETN	Return	$pc := m[sp]; sp := sp + 1$

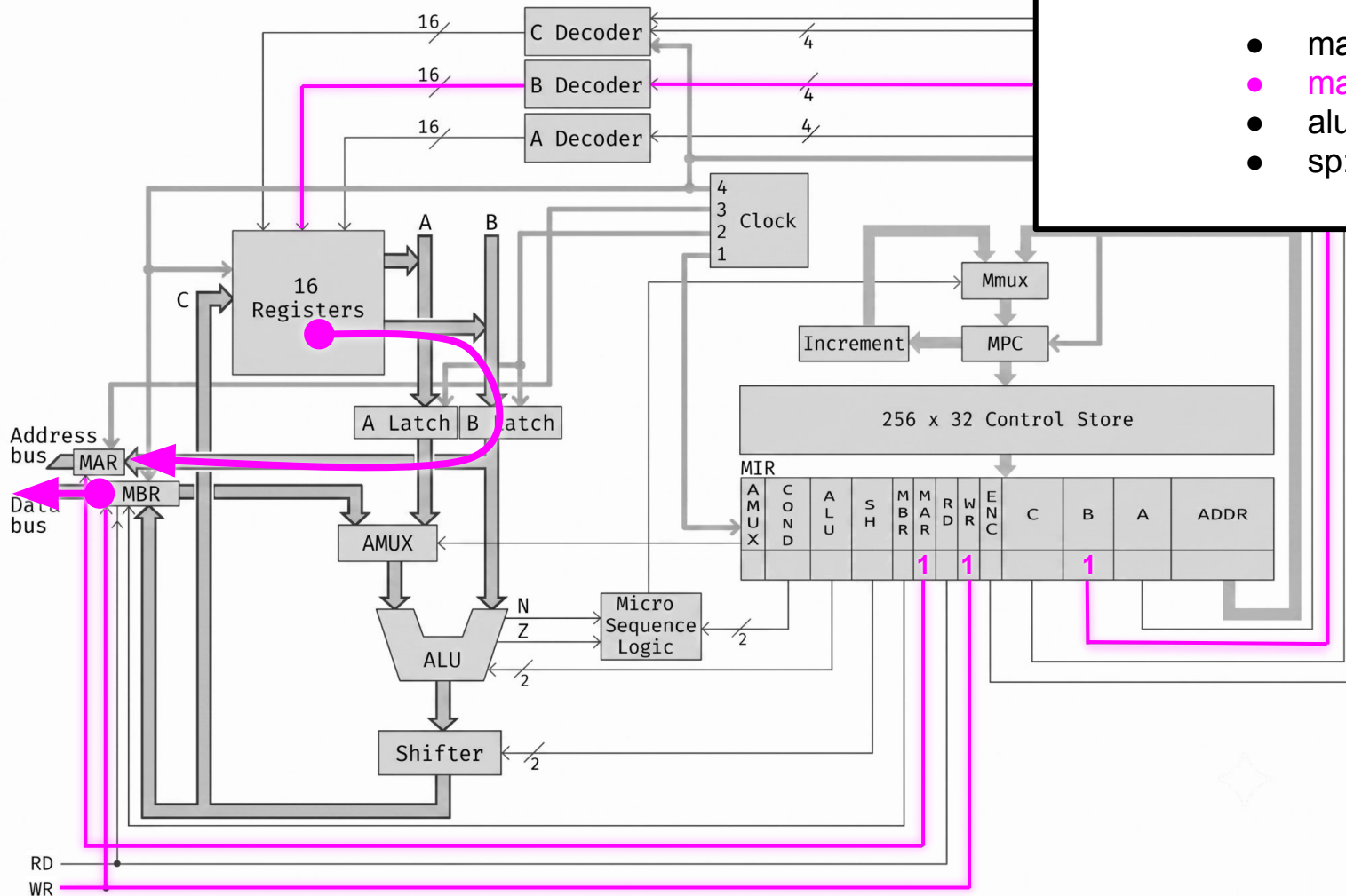
POP INDIRECT

POPI

Saca del tope de la pila y lo guarda en la memoria(ac)

$m[ac] := m[sp]; \quad sp := sp + 1$

- $mar := sp; rd$
- $mar := ac; wr$
- $alu := sp + 1$
- $sp := alu$



Binario	Mnem	Instrucción	Significado
1111000000000000	PSHI	Push indirect	$sp := sp - 1; m[sp] := m[ac]$
1111001000000000	POPI	Pop indirect	$m[ac] := m[sp]; sp := sp + 1$
1111010000000000	PUSH	Push onto stack	$sp := sp - 1; m[sp] := ac$
1111011000000000	POP	Pop from stack	$ac := m[sp]; sp := sp + 1$
1111100000000000	RETN	Return	$pc := m[sp]; sp := sp + 1$

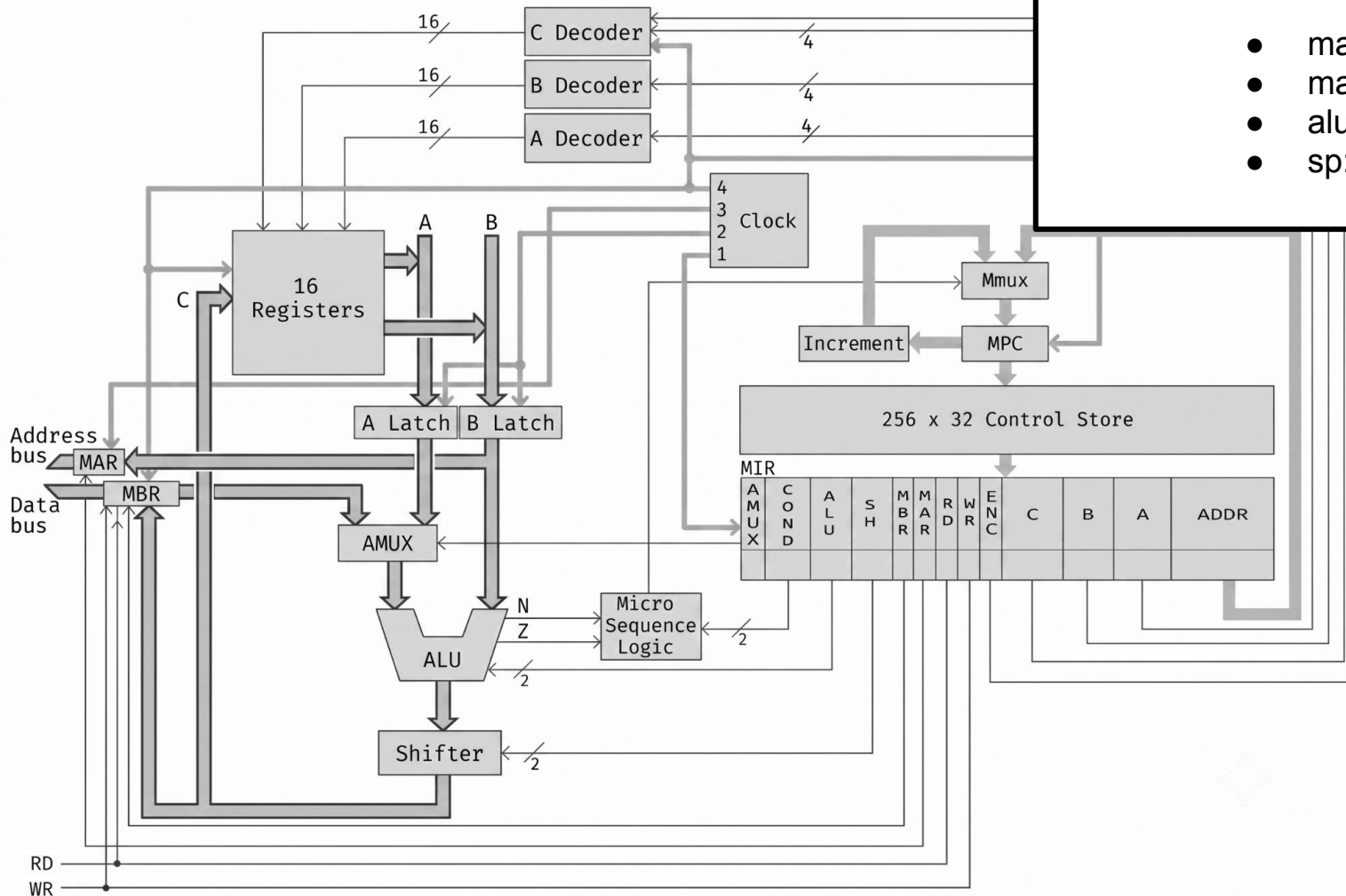
POP INDIRECT

POPI

Saca del tope de la pila y lo guarda en la memoria(ac)

$m[ac] := m[sp]; \quad sp := sp + 1$

- $mar := sp; rd$
- $mar := ac; wr$
- $alu := sp + 1$
- $sp := alu$



Binario	Mnem	Instrucción	Significado
1111000000000000	PSHI	Push indirect	$sp := sp - 1; m[sp] := m[ac]$
1111001000000000	POPI	Pop indirect	$m[ac] := m[sp]; sp := sp + 1$
1111010000000000	PUSH	Push onto stack	$sp := sp - 1; m[sp] := ac$
1111011000000000	POP	Pop from stack	$ac := m[sp]; sp := sp + 1$
1111100000000000	RETN	Return	$pc := m[sp]; sp := sp + 1$

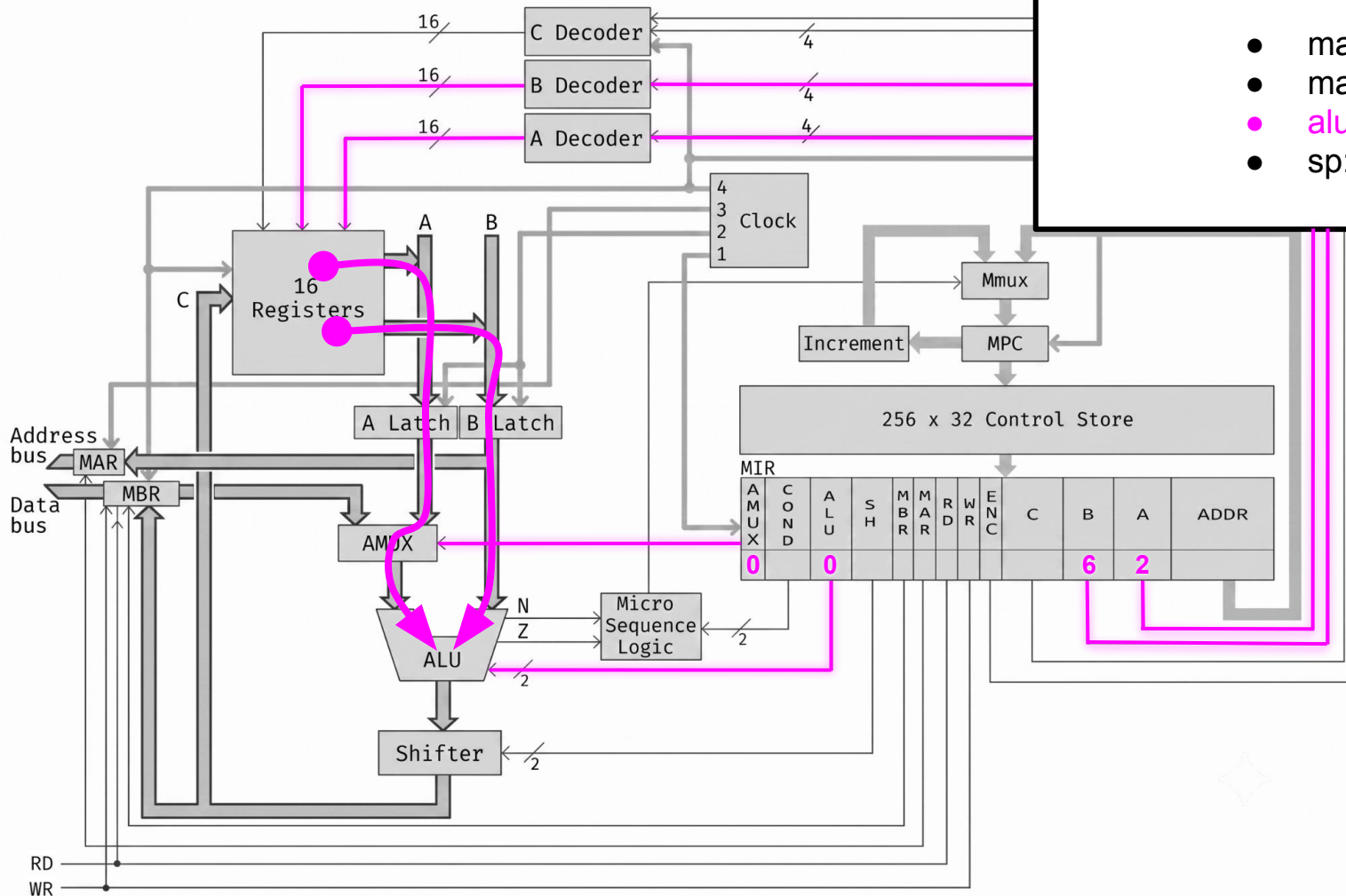
POP INDIRECT

POPI

Saca del tope de la pila y lo guarda en la memoria(ac)

$m[ac] := m[sp]; sp := sp + 1$

- $mar := sp; rd$
- $mar := ac; wr$
- $alu := sp + 1$
- $sp := alu$



Binario	Mnem	Instrucción	Significado
1111000000000000	PSHI	Push indirect	$sp := sp - 1; m[sp] := m[ac]$
1111001000000000	POPI	Pop indirect	$m[ac] := m[sp]; sp := sp + 1$
1111010000000000	PUSH	Push onto stack	$sp := sp - 1; m[sp] := ac$
1111011000000000	POP	Pop from stack	$ac := m[sp]; sp := sp + 1$
1111100000000000	RETN	Return	$pc := m[sp]; sp := sp + 1$

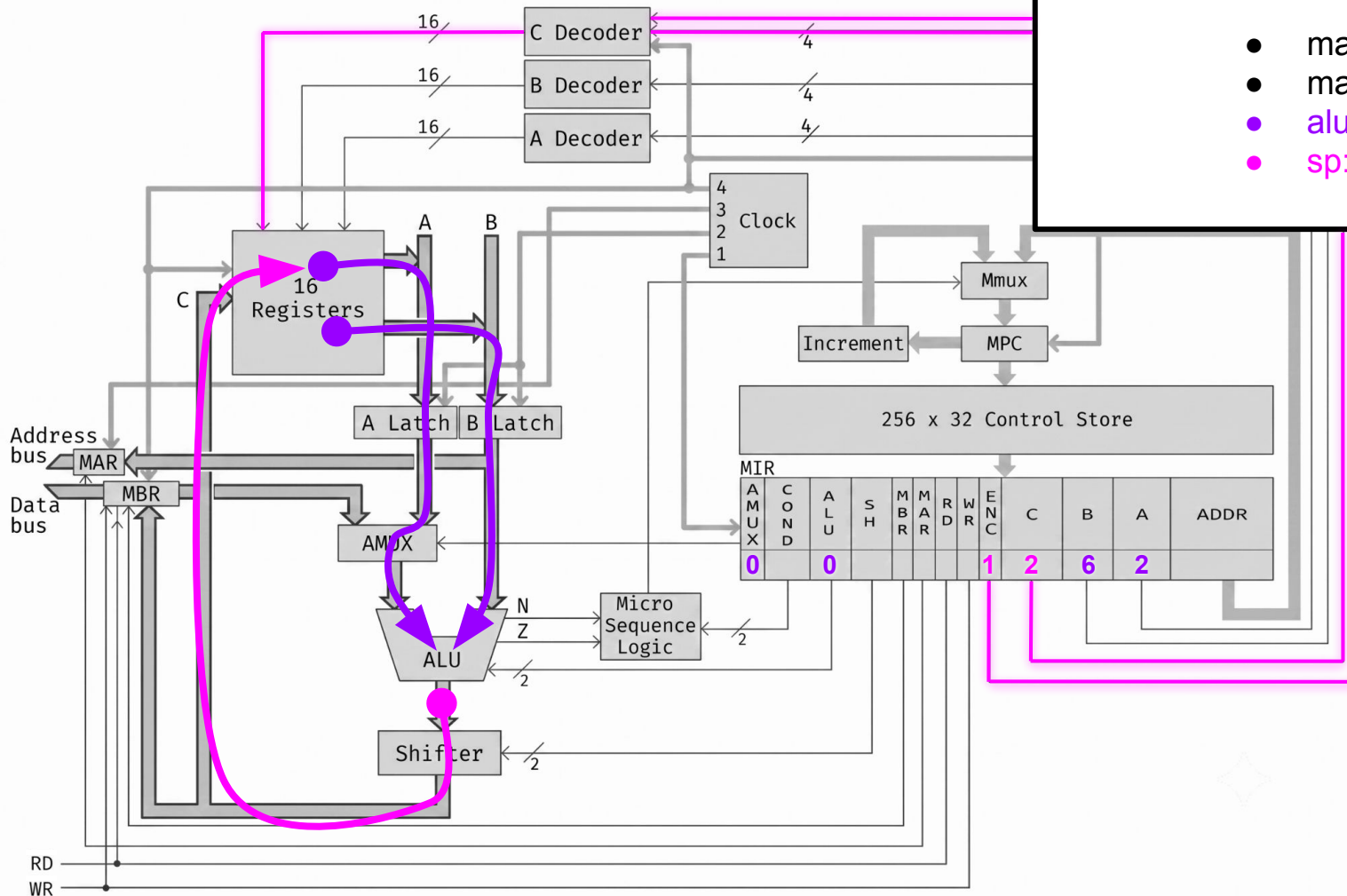
POP INDIRECT

POPI

Saca del tope de la pila y lo guarda en la memoria(ac)

$m[ac] := m[sp]; sp := sp + 1$

- $mar := sp; rd$
- $mar := ac; wr$
- $alu := sp + 1$
- $sp := alu$



Resumiendo

- Una instrucción de C o Pascal se descompone en varias instrucciones assembler
- Una instrucción assembler se descompone en una o varias acciones que debe realizar el micro-procesador
- Una acción se resuelve en una o varias micro-instrucciones
- Cada micro-instrucción consume 1 ciclo del clock

Arquitectura Horizontal

Trayectoria de ejecución



Fundamentos de Arquitectura de computadoras

Análisis de la trayectoria de ejecución horizontal y el flujo de datos dentro del procesador.

- Set de instrucciones
- Ejemplos
- Flujo de datos

